

Phương pháp lập lịch hoạt động tốt. Họ ưu tiên lập lịch theo hàm mũ vì nó dễ điều chỉnh và hội tụ đến giải pháp tối ưu nhanh hơn một chút.

Họ cũng đề cập rằng phương pháp này dễ thực hiện hơn so với lập lịch hiệu năng, nhưng trong Keras cả hai tùy chọn đều dễ dàng. Tuy nhiên, phương pháp 1 chu kỳ dường như cho hiệu suất tốt hơn.

Việc triển khai lập lịch công suất trong Keras là lựa chọn dễ nhất: chỉ cần thiết lập siêu tham số decay khi tạo trình tối ưu hóa:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, decay=1e-4)
```

Hệ số suy giảm là nghịch đảo của s (số bước cần thiết để chia tốc độ học cho thêm một đơn vị), và Keras giả định rằng c bằng 1.

Lập lịch theo hàm mũ và lập lịch từng phần cũng khá đơn giản. Trước tiên, bạn cần định nghĩa một hàm nhận vào epoch hiện tại và trả về tốc độ học. Ví dụ, hãy cùng triển khai lập lịch theo hàm mũ:

```
def exponential_decay_fn(epoch):
    trả về 0.01 * 0.1 ** (thời đại / 20)
```

Nếu bạn không muốn mã hóa cứng các giá trị η và s , bạn có thể tạo một hàm trả về một hàm đã được cấu hình:

```
def exponential_decay(lr0, s):
    def exponential_decay_fn(epoch): return lr0
    * ** (epoch / s) 0.1 return exponential_decay_fn
```

```
exponential_decay_fn = exponential_decay(lr0=0.01, s=20)
```

Tiếp theo, tạo một hàm gọi lại LearningRateScheduler, cung cấp cho nó hàm lập lịch và truyền hàm gọi lại này cho phương thức fit():

```
lr_scheduler =
tf.keras.callbacks.LearningRateScheduler(exponential_decay_fn) history =
model.fit(X_train, y_train, [...], callbacks= [lr_scheduler])
```

LearningRateScheduler sẽ cập nhật thuộc tính `learning_rate` của trình tối ưu hóa vào đầu mỗi epoch. Việc cập nhật tốc độ học một lần mỗi epoch thường là đủ, nhưng nếu bạn muốn cập nhật thường xuyên hơn, ví dụ như ở mỗi bước, bạn luôn có thể viết hàm gọi lại riêng (xem phần “Lập lịch theo cấp số mũ” của notebook để biết ví dụ). Việc cập nhật tốc độ học ở mỗi bước có thể hữu ích nếu có nhiều bước trong mỗi epoch. Ngoài ra, bạn có thể sử dụng phương pháp `tf.keras.optimizers.schedules`, được mô tả ngắn gọn bên dưới.

MẸO

Sau khi huấn luyện, `history.history["lr"]` cho phép bạn truy cập vào danh sách các tốc độ học được sử dụng trong quá trình huấn luyện.

Hàm lập lịch có thể tùy chọn nhận tốc độ học hiện tại làm đối số thứ hai. Ví dụ, hàm lập lịch sau đây nhân tốc độ học trước đó với $0,1$, dẫn đến sự suy giảm theo hàm mũ tương tự (ngoại trừ sự suy giảm giờ bắt đầu từ đầu kỷ nguyên 0 thay vì 1):

```
def exponential_decay_fn(epoch, lr): return lr *
    0,1 ** (1 / 20)
```

Cách triển khai này dựa vào tốc độ học ban đầu của trình tối ưu hóa (ngược lại với cách triển khai trước đó), vì vậy hãy đảm bảo thiết lập tốc độ này một cách phù hợp.

Khi bạn lưu một mô hình, trình tối ưu hóa và tốc độ học của nó cũng được lưu cùng với mô hình đó. Điều này có nghĩa là với hàm lập lịch mới này, bạn chỉ cần tải một mô hình đã được huấn luyện và tiếp tục huấn luyện từ nơi nó đã dừng lại mà không gặp vấn đề gì. Tuy nhiên, mọi thứ không đơn giản như vậy nếu hàm lập lịch của bạn sử dụng đối số epoch: epoch không được lưu lại và nó được đặt lại về 0 mỗi khi bạn gọi phương thức `fit()`. Nếu bạn tiếp tục huấn luyện một mô hình từ nơi nó đã dừng lại, điều này có thể dẫn đến tốc độ học rất lớn, có khả năng làm hỏng trọng số của mô hình. Một giải pháp là đặt thủ công `fit()`.

Tham số `initial_epoch` của phương thức đảm bảo epoch bắt đầu ở giá trị chính xác.

Đối với lập lịch hằng số từng phần, bạn có thể sử dụng một hàm lập lịch như sau (như đã nói ở trên, bạn có thể định nghĩa một hàm tổng quát hơn nếu muốn; xem phần “Lập lịch hằng số từng phần” của sổ tay hướng dẫn để biết ví dụ), sau đó tạo một hàm gọi lại `LearningRateScheduler` với hàm này và truyền nó cho phương thức `fit()`, giống như chúng ta đã làm với lập lịch hàm mũ:

```
def piecewise_constant_fn(epoch): if epoch < 5:
    return 0.01 elif
    epoch < 15:
    return 0.005 else:
    return 0.001
```

Để lập lịch hiệu năng, hãy sử dụng hàm gọi lại `ReduceLROnPlateau`. Ví dụ, nếu bạn truyền hàm gọi lại sau vào phương thức `fit()`, nó sẽ nhân tốc độ học với 0,5 bất cứ khi nào tổn thất xác thực tốt nhất không được cải thiện trong năm epoch liên tiếp (có các tùy chọn khác; vui lòng xem tài liệu để biết thêm chi tiết):

```
lr_scheduler = tf.keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5) history =
model.fit(X_train,
y_train, [...], callbacks= [lr_scheduler])
```

Cuối cùng, Keras cung cấp một cách khác để triển khai lập lịch tốc độ học: bạn có thể định nghĩa tốc độ học theo lịch trình bằng cách sử dụng một trong các lớp có sẵn trong `tf.keras.optimizers.schedules`, sau đó truyền nó cho bất kỳ trình tối ưu hóa nào. Cách tiếp cận này cập nhật tốc độ học ở mỗi bước thay vì ở mỗi epoch. Ví dụ, đây là cách triển khai cùng một lịch trình hàm mũ như hàm `exponential_decay_fn()` mà chúng ta đã định nghĩa trước đó:

nhập toán học

```
batch_size = 32
n_epochs = 25
n_steps = n_epochs * math.ceil(len(X_train) / batch_size)
scheduled_learning_rate =
tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=0.01, decay_steps=n_steps,
    decay_rate=0.1)
optimizer =
tf.keras.optimizers.SGD(learning_rate=scheduled_learning_rate)
```

Cách này rất hay và đơn giản, hơn nữa khi bạn lưu mô hình, tốc độ học và lịch trình của nó (bao gồm cả trạng thái) cũng được lưu lại.

Đối với 1cycle, Keras không hỗ trợ trực tiếp, nhưng có thể triển khai nó chỉ với chưa đến 30 dòng mã bằng cách tạo một hàm callback tùy chỉnh để điều chỉnh tốc độ học ở mỗi lần lặp. Để cập nhật tốc độ học của trình tối ưu hóa từ bên trong phương thức `on_batch_begin()` của hàm callback, bạn cần gọi `tf.keras.backend.set_value(self.model.optimizer.learning_rate, new_learning_rate)`. Xem phần "Lập lịch 1Cycle" trong notebook để biết ví dụ.

Tóm lại, suy giảm theo cấp số mũ, lập lịch hiệu năng và 1 chu kỳ có thể tăng tốc độ hội tụ đáng kể, vì vậy hãy thử chúng!

Tránh hiện tượng quá khớp dữ liệu thông qua phương pháp điều chỉnh (Regularization).

Với bốn thông số, tôi có thể tạo hình một con voi, và với năm thông số, tôi có thể làm cho nó ngoe nguẩy cái vòi của mình.

—John von Neumann, được Enrico Fermi trích dẫn trong Nature 427

Với hàng ngàn tham số, bạn có thể bao quát toàn bộ "vườn thú". Mạng nơ-ron sâu thường có hàng chục ngàn tham số, đôi khi thậm chí là hàng triệu. Điều này mang lại cho chúng sự tự do đáng kinh ngạc và có nghĩa là chúng có thể phù hợp với rất nhiều tập dữ liệu phức tạp. Nhưng sự linh hoạt tuyệt vời này cũng dẫn đến những hạn chế.

Điều này khiến mạng dễ bị quá khớp với tập dữ liệu huấn luyện. Việc điều chỉnh (regularization) thường cần thiết để ngăn chặn điều này.

Chúng ta đã triển khai một trong những kỹ thuật điều chỉnh tốt nhất trong **Chương 10**: dừng sớm. Hơn nữa, mặc dù Batch Normalization được thiết kế để giải quyết vấn đề gradient không ổn định, nó cũng hoạt động như một bộ điều chỉnh khá tốt. Trong phần này, chúng ta sẽ xem xét các kỹ thuật điều chỉnh phổ biến khác cho mạng nơ-ron: điều chỉnh ℓ_1 và ℓ_2 , dropout và điều chỉnh max-norm.

ℓ_1 và ℓ_2 Điều chỉnh

Tương tự như cách bạn đã làm ở **Chương 4** với các mô hình tuyến tính đơn giản, bạn có thể sử dụng chuẩn hóa ℓ_1 để ràng buộc trọng số kết nối của mạng nơ-ron, và/hoặc chuẩn hóa ℓ_2 nếu bạn muốn một mô hình thưa (với nhiều trọng số bằng 0). Dưới đây là cách áp dụng chuẩn hóa ℓ_1 cho trọng số kết nối của một lớp Keras, sử dụng hệ số chuẩn hóa là 0,01:

```
layer = tf.keras.layers.Dense(100, activation="relu",
                               kernel_initializer="he_normal",
                               kernel_regularizer=tf.keras.regularizers.l2(0.01))
```

Hàm `l2()` trả về một bộ điều chỉnh sẽ được gọi ở mỗi bước trong quá trình huấn luyện để tính toán tổn thất điều chỉnh. Sau đó, tổn thất này được cộng vào tổn thất cuối cùng. Như bạn có thể đoán, bạn chỉ cần sử dụng `tf.keras.regularizers.l1()` nếu muốn điều chỉnh ℓ_1 ; nếu muốn cả điều chỉnh ℓ_1 và ℓ_2 , hãy sử dụng `tf.keras.regularizers.l1_l2()` (chỉ định cả hai hệ số điều chỉnh).

Vì thông thường bạn sẽ muốn áp dụng cùng một bộ điều chỉnh cho tất cả các lớp trong mạng của mình, cũng như sử dụng cùng một hàm kích hoạt và cùng một chiến lược khởi tạo trong tất cả các lớp ẩn, bạn có thể thấy mình lặp lại các tham số giống nhau. Điều này làm cho mã trở nên khó đọc và dễ xảy ra lỗi. Để tránh điều này, bạn có thể thử viết lại mã của mình để sử dụng vòng lặp. Một lựa chọn khác là sử dụng

Hàm `functools.partial()` của Python cho phép bạn tạo một lớp bao bọc mỏng cho bất kỳ hàm nào có thể gọi được, với một số giá trị tham số mặc định:

```
from functools import partial

RegularizedDense = partial(tf.keras.layers.Dense,
                           activation="relu",
                           kernel_initializer="he_normal",
                           kernel_regularizer=tf.keras.regularizers.l2(0.01))

model =
    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=[28,
28]),
    RegularizedDense(100),
    RegularizedDense(100), RegularizedDense(10, activation="softmax")
])
```

CẢNH BÁO

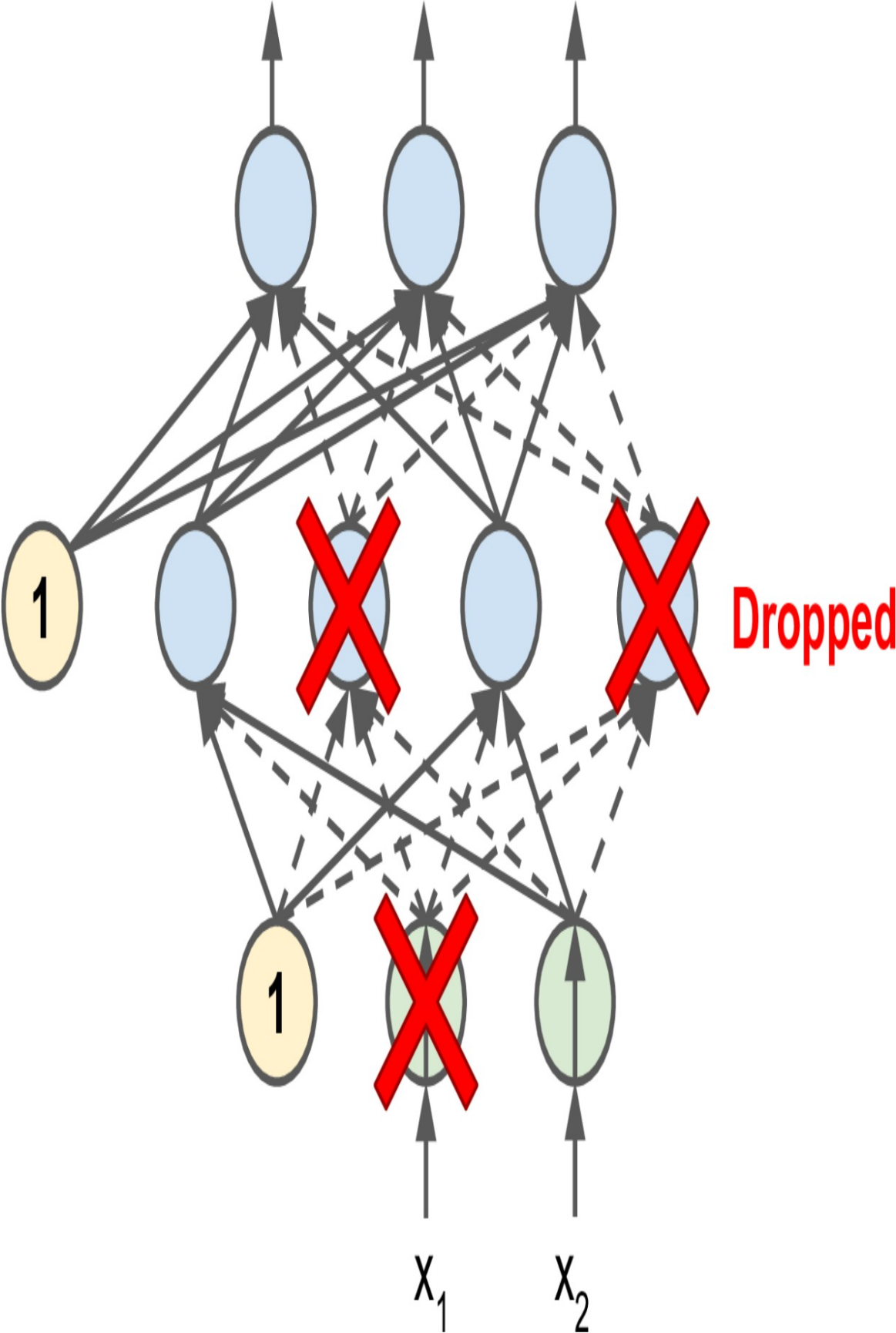
Như chúng ta đã thấy trước đó, chuẩn hóa ℓ_2 hoạt động tốt khi sử dụng SGD, tối ưu hóa động lượng và tối ưu hóa động lượng Nesterov, nhưng không phù hợp với Adam và các biến thể của nó. Nếu bạn muốn sử dụng Adam với suy giảm trọng số, thì không nên sử dụng chuẩn hóa ℓ_2 : hãy sử dụng AdamW thay thế.

Bỏ học

Dropout là một trong những kỹ thuật điều chỉnh phổ biến nhất cho mạng nơ-ron sâu. Nó được đề xuất trong một bài báo²⁶ bởi Geoffrey Hinton vào năm 2012 và được trình bày chi tiết hơn trong một bài báo năm 2014²⁷. Phương pháp dropout được Nitish Srivastava và cộng sự đề xuất và đã chứng minh được sự thành công vượt trội: nhiều mạng nơ-ron hiện đại sử dụng dropout vì nó giúp tăng độ chính xác lên 1-2%. Con số này có vẻ không nhiều, nhưng khi một mô hình đã có độ chính xác 95%, việc tăng độ chính xác thêm 2% đồng nghĩa với việc giảm tỷ lệ lỗi xuống gần 40% (từ 5% lỗi xuống khoảng 3%).

Đây là một thuật toán khá đơn giản: ở mỗi bước huấn luyện, mỗi nơ-ron (bao gồm cả nơ-ron đầu vào, nhưng luôn loại trừ nơ-ron đầu ra) đều có xác suất p bị "loại bỏ" tạm thời, nghĩa là nó sẽ bị bỏ qua hoàn toàn trong bước huấn luyện này, nhưng nó có thể hoạt động trong các bước tiếp theo.

Bước tiếp theo (xem **Hình 11-10**). Siêu tham số p được gọi là tỷ lệ bỏ qua (dropout rate), và nó thường được đặt trong khoảng từ 10% đến 50%: gần hơn với 20-30% trong mạng nơ-ron hồi quy (xem **Chương 15**), và gần hơn với 40-50% trong mạng nơ-ron tích chập (xem **Chương 14**). Sau khi huấn luyện, các nơ-ron sẽ không bị bỏ qua nữa. Và đó là tất cả (ngoại trừ một chi tiết kỹ thuật mà chúng ta sẽ thảo luận ngay sau đây).



Hình 11-10. Với phương pháp điều chỉnh dropout, ở mỗi lần lặp huấn luyện, một tập hợp con ngẫu nhiên của tất cả các nơ-ron trong một hoặc nhiều lớp-ngoại trừ lớp đầu ra-được "loại bỏ"; các nơ-ron này xuất ra giá trị 0 ở lần lặp này (được biểu thị bằng các mũi tên nét đứt).

Thoạt đầu, thật đáng ngạc nhiên khi kỹ thuật phá hoại này lại có hiệu quả. Liệu một công ty có hoạt động tốt hơn nếu nhân viên được yêu cầu tung đồng xu mỗi sáng để quyết định có đi làm hay không? Ai biết được; có lẽ là có! Công ty sẽ buộc phải thích nghi với cơ cấu tổ chức của mình; họ không thể chỉ dựa vào một người để vận hành máy pha cà phê hoặc thực hiện bất kỳ nhiệm vụ quan trọng nào khác, vì vậy chuyên môn này sẽ phải được phân bổ cho nhiều người. Nhân viên sẽ phải học cách hợp tác với nhiều đồng nghiệp, chứ không chỉ một vài người. Công ty sẽ trở nên kiên cường hơn nhiều. Nếu một người nghỉ việc, điều đó sẽ không tạo ra nhiều khác biệt.

Chưa rõ liệu ý tưởng này có thực sự hiệu quả đối với các công ty hay không, nhưng chắc chắn nó có hiệu quả đối với mạng nơ-ron. Các nơ-ron được huấn luyện bằng kỹ thuật dropout không thể thích nghi cùng với các nơ-ron lân cận; chúng phải tự hoạt động hiệu quả nhất có thể. Chúng cũng không thể phụ thuộc quá nhiều vào chỉ một vài nơ-ron đầu vào; chúng phải chú ý đến từng nơ-ron đầu vào của mình. Kết quả là, chúng ít nhạy cảm hơn với những thay đổi nhỏ trong đầu vào. Cuối cùng, bạn sẽ có được một mạng lưới mạnh mẽ hơn và có khả năng khái quát hóa tốt hơn.

Một cách khác để hiểu sức mạnh của dropout là nhận ra rằng một mạng nơ-ron độc nhất được tạo ra ở mỗi bước huấn luyện. Vì mỗi nơ-ron có thể hiện diện hoặc vắng mặt, nên tổng cộng có 2 mạng khả dĩ (trong đó N là tổng số nơ-ron có thể bị loại bỏ). Con số này lớn đến mức hầu như không thể có cùng một mạng nơ-ron được lấy mẫu hai lần.

Sau khi thực hiện 10.000 bước huấn luyện, về cơ bản bạn đã huấn luyện 10.000 mạng nơ-ron khác nhau, mỗi mạng chỉ được huấn luyện một lần.

Rõ ràng là các mạng nơ-ron này không độc lập vì chúng chia sẻ nhiều trọng số, nhưng dù vậy chúng vẫn khác nhau. Mạng nơ-ron kết quả có thể được xem như một tập hợp trung bình của tất cả các mạng nơ-ron nhỏ hơn này.

MẸO

Trên thực tế, bạn thường chỉ có thể áp dụng dropout cho các neuron ở một đến ba lớp trên cùng (không bao gồm lớp đầu ra).

Có một chi tiết kỹ thuật nhỏ nhưng quan trọng. Giả sử $p = 75\%$: trung bình chỉ có 25% tổng số nơ-ron hoạt động ở mỗi bước trong quá trình huấn luyện. Điều này có nghĩa là sau khi huấn luyện, một nơ-ron sẽ được kết nối với số lượng nơ-ron đầu vào gấp bốn lần so với trong quá trình huấn luyện. Để bù đắp cho điều này, chúng ta cần nhân trọng số kết nối đầu vào của mỗi nơ-ron với bốn trong quá trình huấn luyện. Nếu không, mạng nơ-ron sẽ không hoạt động tốt vì nó sẽ nhận được dữ liệu khác nhau trước và sau khi huấn luyện. Nói chung hơn, chúng ta cần chia trọng số kết nối cho xác suất giữ lại ($1 - p$) trong quá trình huấn luyện.

Để triển khai dropout bằng Keras, bạn có thể sử dụng lớp `tf.keras.layers.Dropout`. Trong quá trình huấn luyện, nó sẽ ngẫu nhiên loại bỏ một số đầu vào (đặt chúng về 0) và chia các đầu vào còn lại cho xác suất giữ lại. Sau khi huấn luyện, nó không làm gì cả; nó chỉ chuyển các đầu vào đến lớp tiếp theo. Đoạn mã sau áp dụng chuẩn hóa dropout trước mỗi lớp dense, sử dụng tỷ lệ dropout là 0.2:

```
model =
    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=[28,
28]), tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(100, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(rate=0.2),
    tf.keras.layers.Dense(10, activation="softmax")

]) [...] # biên dịch và huấn luyện mô hình
```

CẢNH BÁO

Vì dropout chỉ hoạt động trong quá trình huấn luyện, việc so sánh tổn thất huấn luyện và tổn thất xác thực có thể gây hiểu nhầm. Cụ thể, một mô hình có thể bị overfitting trên tập huấn luyện nhưng lại có tổn thất huấn luyện và xác thực tương tự nhau. Vì vậy, hãy đảm bảo đánh giá tổn thất huấn luyện mà không sử dụng dropout (ví dụ: sau khi huấn luyện).

Nếu bạn nhận thấy mô hình bị quá khớp (overfitting), bạn có thể tăng tỷ lệ dropout. Ngược lại, bạn nên thử giảm tỷ lệ dropout nếu mô hình không đủ khớp (underfitting) với tập dữ liệu huấn luyện. Việc tăng tỷ lệ dropout cho các lớp lớn và giảm cho các lớp nhỏ cũng có thể hữu ích. Hơn nữa, nhiều kiến trúc hiện đại chỉ sử dụng dropout sau lớp ẩn cuối cùng, vì vậy bạn có thể thử cách này nếu dropout toàn bộ quá mạnh.

Việc sử dụng kỹ thuật dropout có xu hướng làm chậm quá trình hội tụ đáng kể, nhưng nó thường mang lại mô hình tốt hơn khi được tinh chỉnh đúng cách. Vì vậy, nhìn chung, nó rất đáng để bỏ thêm thời gian và công sức, đặc biệt là đối với các mô hình lớn.

MẸO

Nếu bạn muốn chuẩn hóa một mạng tự chuẩn hóa dựa trên hàm kích hoạt SELU (như đã thảo luận trước đó), bạn nên sử dụng alpha dropout: đây là một biến thể của dropout giúp bảo toàn giá trị trung bình và độ lệch chuẩn của các đầu vào. Nó được giới thiệu trong cùng bài báo với SELU, vì dropout thông thường sẽ phá vỡ tính tự chuẩn hóa.

Bỏ học Monte Carlo (MC)

Năm 2016, một bài báo²⁸ Yarin Gal và Zoubin Ghahramani đã bổ sung thêm một vài lý do thuyết phục nữa để sử dụng dropout:

- Đầu tiên, bài báo đã thiết lập mối liên hệ sâu sắc giữa mạng nơ-ron có lớp bỏ qua (tức là mạng nơ-ron chứa các lớp bỏ qua) và suy luận Bayes xấp xỉ, mang lại cơ sở toán học vững chắc cho việc sử dụng lớp bỏ qua.²⁹

- Thứ hai, các tác giả đã giới thiệu một kỹ thuật mạnh mẽ gọi là MC.

Dropout, có thể nâng cao hiệu suất của bất kỳ dropout nào đã được huấn luyện.

mô hình mà không cần phải huấn luyện lại hoặc thậm chí sửa đổi nó chút nào. Nó cũng

cung cấp một thước đo tốt hơn nhiều về sự không chắc chắn của mô hình, và nó có thể

Có thể được triển khai chỉ với vài dòng mã.

Nếu tất cả những điều này nghe có vẻ giống như một chiêu trò câu view "một mảnh khố kỳ quặc", thì hãy xem...

Đoạn mã sau đây. Đây là bản triển khai đầy đủ của MC Dropout, boosting.

Mô hình bỏ qua (dropout model) mà chúng ta đã huấn luyện trước đó mà không cần huấn luyện lại:

```
import numpy as np

y_probas = np.stack([model(X_test, training=True)
                     cho mẫu trong phạm vi (100)])
y_proba = y_probas.mean(axis=0)
```

Lưu ý rằng model(X) tương tự như model.predict(X) ngoại trừ việc nó trả về

một tensor thay vì một mảng NumPy, và nó hỗ trợ quá trình huấn luyện.

đối số. Trong ví dụ mã này, việc đặt training=True đảm bảo rằng

Lớp Dropout vẫn hoạt động, vì vậy tất cả các dự đoán sẽ hơi khác nhau. Chúng tôi

Chỉ cần đưa ra 100 dự đoán trên tập dữ liệu thử nghiệm, và chúng ta sẽ tính trung bình của chúng.

Cụ thể hơn, mỗi lần gọi đến mô hình sẽ trả về một ma trận với một hàng tương ứng với mỗi phần tử được gán.

mỗi lớp có một cột riêng. Vì có 10.000 trường hợp trong

Tập dữ liệu thử nghiệm và 10 lớp, đây là một ma trận có kích thước [10000, 10]. Chúng ta xếp chồng lên nhau.

Có 100 ma trận như vậy, do đó y_probas là một mảng 3D có kích thước [100, 10000, 10].

Khi ta lấy trung bình trên chiều thứ nhất (trục = 0), ta được y_proba, một

mảng có kích thước [10000, 10], giống như chúng ta nhận được với một dự đoán duy nhất.

Vậy là xong! Tính trung bình của nhiều dự đoán với tính năng dropout được bật sẽ cho kết quả như sau:

Chúng tôi sử dụng phương pháp ước tính Monte Carlo, phương pháp này thường đáng tin cậy hơn kết quả của một phương pháp khác.

Dự đoán đơn lẻ với dropout tắt. Ví dụ, hãy xem xét

Dự đoán của mô hình cho trường hợp đầu tiên trong tập dữ liệu thử nghiệm Fashion MNIST, với

Chế độ bỏ học đã tắt:

```
>>> model.predict(X_test[:1]).round(3)
mảng([[0. , 0. 0.      0.      , 0.      , 0.      ,      , 0,024, 0.      , 0.132,
        ,
        0.844]], dtype=float32)
```

Mô hình khá tự tin (84,4%) rằng hình ảnh này thuộc lớp 9.

(giày boot cổ thấp). Hãy so sánh điều này với dự đoán bỏ cuộc của MC:

```
>>> y_proba[0].round(3)
mảng([0. , 0.001, 0. , 0. , 0. , 0. , 0,067, 0. , 0,209,
       0.723], dtype=float32)
```

Mô hình dường như vẫn ưu tiên lớp 9, nhưng độ tin cậy của nó đã giảm xuống.

72,3%, và xác suất ước tính cho các loại giày dép hạng 5 (dép xăng đan) và hạng 7 (giày thể thao).

Số lượng đã tăng lên, điều này hợp lý vì chúng cũng là giày dép.

Phương pháp MC Dropout có xu hướng cải thiện độ tin cậy của xác suất mô hình.

ước tính. Điều này có nghĩa là nó ít có khả năng tự tin nhưng lại sai, điều đó

Điều này có thể rất nguy hiểm: hãy tưởng tượng một chiếc xe tự lái tự tin bỏ qua biển báo dừng.

ký tên. Việc biết chính xác những lớp nào khác có khả năng xảy ra nhất cũng rất hữu ích.

Và bạn cũng có thể xem xét **độ lệch chuẩn của xác suất**.

ước tính:

```
>>> y_std = y_probab.std(axis=0)
>>> y_std[0].round(3)
mảng([0. , 0. 0. 0.001, , , 0.001, 0. , 0,096, 0. , 0,162,
       0.183], dtype=float32)
```

Rõ ràng là có khá nhiều sự khác biệt trong các ước tính xác suất cho

Lớp 9: Độ lệch chuẩn là 0,183, cần được so sánh với...

Xác suất ước tính là 0,723: nếu bạn đang xây dựng một hệ thống nhạy cảm với rủi ro.

(ví dụ: một hệ thống y tế hoặc tài chính), bạn có thể sẽ đối xử với hệ thống đó như vậy.

Dự đoán không chắc chắn với sự thận trọng tối đa. Bạn chắc chắn sẽ không coi nó như một dự đoán không chắc chắn.

Tương tự như dự đoán với độ tin cậy 84,4%. Hơn nữa, độ chính xác của mô hình đã đạt được...

Tăng nhẹ từ 87,0% lên 87,2%:

```
>>> y_pred = y_proba.argmax(axis=1)
>>> độ chính xác = (y_pred == y_test).sum() / len(y_test)
>>> độ chính xác
0.8717
```

GHI CHÚ

Số lượng mẫu Monte Carlo bạn sử dụng (100 trong ví dụ này) là một siêu tham số bạn có thể điều chỉnh. Số lượng càng cao, dự đoán càng chính xác và ước tính độ bất định càng cao. Tuy nhiên, nếu bạn tăng gấp đôi, thời gian suy luận cũng sẽ tăng gấp đôi. Hơn nữa, khi số lượng mẫu vượt quá một mức nhất định, bạn sẽ nhận thấy rất ít sự cải thiện. Vì vậy, nhiệm vụ của bạn là tìm ra sự cân bằng phù hợp giữa độ trễ và độ chính xác, tùy thuộc vào ứng dụng của bạn.

Nếu mô hình của bạn chứa các lớp khác hoạt động theo cách đặc biệt trong quá trình huấn luyện (chẳng hạn như các lớp BatchNormalization), thì bạn không nên ép buộc chế độ huấn luyện như chúng ta vừa làm. Thay vào đó, bạn nên thay thế các lớp Dropout bằng lớp MCDropout sau:

```
class MCDropout(tf.keras.layers.Dropout):
    def call(self, inputs, training=False):
        return super().call(inputs, training=True)
```

Ở đây, chúng ta chỉ cần tạo lớp con của lớp Dropout và ghi đè phương thức call() để buộc tham số training của nó thành True (xem [Chương 12](#)). Tương tự, bạn có thể định nghĩa một lớp MCAphaDropout bằng cách kế thừa từ AlphaDropout. Nếu bạn đang tạo một mô hình từ đầu, chỉ cần sử dụng MCDropout thay vì Dropout. Nhưng nếu bạn có một mô hình đã được huấn luyện bằng Dropout, bạn cần tạo một mô hình mới giống hệt mô hình hiện có ngoại trừ việc sử dụng Dropout thay vì MCDropout, sau đó sao chép trọng số của mô hình hiện có sang mô hình mới của bạn.

Tóm lại, MC Dropout là một kỹ thuật tuyệt vời giúp tăng cường hiệu quả của các mô hình dropout và cung cấp ước lượng độ bất định tốt hơn. Và tất nhiên, vì nó chỉ là dropout thông thường trong quá trình huấn luyện, nên nó cũng hoạt động như một bộ điều chỉnh (regularizer).

Chuẩn hóa Max-Norm

Một kỹ thuật điều chỉnh phổ biến khác cho mạng nơ-ron được gọi là điều chỉnh chuẩn tối đa (max-norm regularization): đối với mỗi nơ-ron, nó ràng buộc các trọng số w của...

các kết nối đến sao cho $w \leq r$, trong đó r là siêu tham số chuẩn tối đa và $\cdot$ là chuẩn ℓ_2 .

Phương pháp chuẩn hóa max-norm không thêm một số hạng mất mát chuẩn hóa vào hàm mất mát tổng thể. Thay vào đó, nó thường được thực hiện bằng cách tính toán $\|w\|_2$ sau mỗi bước huấn luyện và điều chỉnh lại w nếu cần ($w \leftarrow w \cdot r / \|w\|_2$).

Việc giảm r sẽ làm tăng lượng chuẩn hóa và giúp giảm hiện tượng quá khớp. Chuẩn hóa max-norm cũng có thể giúp giảm bớt các vấn đề về độ dốc không ổn định (nếu bạn không sử dụng Batch Normalization).

Để triển khai chuẩn hóa max-norm trong Keras, hãy đặt đối số `kernel_constraint` của mỗi lớp ẩn thành ràng buộc `max_norm()` với giá trị max thích hợp, như sau:

```
dense = tf.keras.layers.Dense(
    100, activation="relu", kernel_initializer="he_normal",
    kernel_constraint=tf.keras.constraints.max_norm(1.))
```

Sau mỗi lần lặp huấn luyện, phương thức `fit()` của mô hình sẽ gọi đối tượng được trả về bởi `max_norm()`, truyền vào đó trọng số của lớp và nhận lại trọng số đã được điều chỉnh tỷ lệ, sau đó trọng số này sẽ thay thế trọng số của lớp. Như bạn sẽ thấy trong [Chương 12](#), bạn có thể định nghĩa hàm ràng buộc tùy chỉnh của riêng mình nếu cần và sử dụng nó làm `kernel_constraint`. Bạn cũng có thể ràng buộc các số hạng bias bằng cách đặt đối số `bias_constraint`.

Hàm `max_norm()` có một đối số `axis` mặc định là 0. Một lớp Dense thường có trọng số dạng [số lượng đầu vào, số lượng nơ-ron], vì vậy việc sử dụng `axis=0` có nghĩa là ràng buộc max-norm sẽ được áp dụng độc lập cho vectơ trọng số của mỗi nơ-ron. Nếu bạn muốn sử dụng max-norm với các lớp tích chập (xem [Chương 14](#)), hãy đảm bảo đặt đối số `axis` của ràng buộc `max_norm()` một cách thích hợp (thường là `axis=[0, 1, 2]`).

Tóm tắt và hướng dẫn thực hành

Trong chương này, chúng ta đã đề cập đến nhiều kỹ thuật khác nhau, và bạn có thể đang tự hỏi nên sử dụng kỹ thuật nào. Điều này phụ thuộc vào nhiệm vụ cụ thể, và hiện vẫn chưa có sự đồng thuận rõ ràng, nhưng tôi thấy cấu hình trong **Bảng 11-3** hoạt động tốt trong hầu hết các trường hợp, mà không cần phải tính chỉnh nhiều siêu tham số.

Tuy nhiên, xin đừng coi những cài đặt mặc định này là những quy tắc bất di bất dịch!

T

Một

b

le

1

1

-

3

.

D

ef

Một

bạn

lt

D

N

N

C

O

N

quá

sung

bạn

r

Một

ti

O

N

Giá trị mặc định của siêu tham số

Trình khởi tạo hạt nhân

Sự khởi tạo của anh ấy

Hàm kích hoạt ReLU nếu nông; Swish nếu sâu.	
Chuẩn hóa	Không cần nếu nông; cần Batch Norm nếu sâu.
Điều chỉnh	Dừng sớm; Giảm cân nếu cần thiết.
Trình tối ưu hóa	Phương pháp gia tốc gradient Nesterov hay AdamW
Lịch trình tốc độ học tập Lịch trình hiệu suất hoặc 1 chu kỳ	

Nếu mạng nơ-ron chỉ là một chồng các lớp dày đặc đơn giản, thì nó có thể tự chuẩn hóa.
và bạn nên sử dụng cấu hình trong **Bảng 11-4** thay thế.

Tab

1e11-4

•

DNIcon

Figure

Figure

el

chủ đề 1

Một

lý tử

N

g

N

và

Giá trị mặc định của siêu tham số

Trình khởi tạo hạt nhân	Khởi tạo LeCun
Chức năng kích hoạt SELU	
Chuẩn hóa	Không có (tự chuẩn hóa)
Điều chỉnh	Bỏ học Alpha nếu cần thiết
Trình tối ưu hóa	Độ dốc gia tốc Nesterov
Lịch trình tốc độ học tập Lịch trình hiệu suất hoặc 1 chu kỳ	

Đừng quên chuẩn hóa các đặc trưng đầu vào! Bạn cũng nên cố gắng tái sử dụng các phần của mạng nơ-ron đã được huấn luyện trước nếu bạn tìm thấy một mạng giải quyết được vấn đề tương tự, hoặc sử dụng huấn luyện trước không giám sát nếu bạn có nhiều dữ liệu chưa được gắn nhãn, hoặc sử dụng huấn luyện trước trên một tác vụ phụ trợ nếu bạn có nhiều dữ liệu đã được gắn nhãn cho một tác vụ tương tự.

Mặc dù các hướng dẫn trước đó bao gồm hầu hết các trường hợp, nhưng vẫn có một số ngoại lệ:

- Nếu bạn cần một mô hình thưa, bạn có thể sử dụng chuẩn hóa ℓ_1 (và tùy chọn đặt các trọng số nhỏ về 0 sau khi huấn luyện). Nếu bạn cần một mô hình thưa hơn nữa, bạn có thể sử dụng Tối ưu hóa mô hình TensorFlow.

Bộ công cụ. Việc này sẽ phá vỡ quá trình tự chuẩn hóa, vì vậy trong trường hợp này bạn nên sử dụng cấu hình mặc định.

- Nếu bạn cần một mô hình có độ trễ thấp (một mô hình thực hiện dự đoán cực nhanh), bạn có thể cần sử dụng ít lớp hơn, sử dụng hàm kích hoạt nhanh như ReLU hoặc leaky ReLU, và gộp các lớp Batch Normalization vào các lớp trước đó sau khi huấn luyện. Việc có một mô hình thưa cũng sẽ giúp ích. Cuối cùng, bạn có thể muốn giảm độ chính xác của số thực từ 32 bit xuống 16 hoặc thậm chí 8 bit (xem “[Triển khai mô hình lên thiết bị di động hoặc thiết bị nhúng](#)”). Một lần nữa, hãy xem TF-MOT.
- Nếu bạn đang xây dựng một ứng dụng nhạy cảm với rủi ro, hoặc độ trễ suy luận không quá quan trọng trong ứng dụng của bạn, bạn có thể sử dụng MC Dropout để tăng hiệu suất và nhận được các ước tính xác suất đáng tin cậy hơn, cùng với các ước tính về độ bất định.

Với những hướng dẫn này, giờ đây bạn đã sẵn sàng huấn luyện các mạng nơ-ron sâu! Tôi hy vọng bạn đã tin tưởng rằng mình có thể tiến rất xa chỉ với API Keras tiện lợi. Tuy nhiên, có thể sẽ có lúc bạn cần kiểm soát nhiều hơn; ví dụ, để viết một hàm mất mát tùy chỉnh hoặc để tinh chỉnh thuật toán huấn luyện. Trong những trường hợp như vậy, bạn sẽ cần sử dụng API cấp thấp hơn của TensorFlow, như bạn sẽ thấy trong chương tiếp theo.

Bài tập

1. Vấn đề mà phương pháp khởi tạo Glorot và phương pháp khởi tạo He hướng đến là gì?
Để sửa chữa?
2. Liệu có được phép khởi tạo tất cả các trọng số bằng cùng một giá trị miễn là giá trị đó được chọn ngẫu nhiên bằng phương pháp khởi tạo He không?
3. Có được phép khởi tạo các hệ số thiên vị bằng 0 không?
4. Trong những trường hợp nào bạn muốn sử dụng từng hàm kích hoạt mà chúng ta đã thảo luận trong chương này?

5. Điều gì có thể xảy ra nếu bạn đặt tham số siêu tham số động lượng quá gần nhau?
thành 1 (ví dụ: 0.99999) khi sử dụng trình tối ưu hóa SGD?
6. Hãy nêu ba cách bạn có thể tạo ra một mô hình thưa.
7. Việc sử dụng dropout có làm chậm quá trình huấn luyện không? Có làm chậm quá trình suy luận (tức là đưa ra dự đoán trên các trường hợp mới) không? Còn về MC Dropout thì sao?
8. Thực hành huấn luyện mạng nơ-ron sâu trên tập dữ liệu hình ảnh CIFAR10:
 - a. Xây dựng một mạng nơ-ron sâu (DNN) với 20 lớp ẩn, mỗi lớp có 100 nơ-ron (tức là...) (Quá nhiều, nhưng đó là mục đích của bài tập này). Sử dụng khởi tạo He và hàm kích hoạt Swish.
 - b. Sử dụng thuật toán tối ưu Nadam và dừng sớm để huấn luyện mạng.
Trên tập dữ liệu CIFAR10. Bạn có thể tải nó bằng lệnh
``tf.keras.datasets.cifar10.load_data()``. Tập dữ liệu bao gồm 60.000 ảnh
màu 32×32 pixel (50.000 ảnh để huấn luyện, 10.000 ảnh để kiểm tra)
với 10 lớp, vì vậy bạn sẽ cần một lớp đầu ra softmax với 10 nơ-ron. Hãy nhớ tìm
kiểm tốc độ học phù hợp mỗi khi bạn thay đổi kiến trúc hoặc siêu tham số của mô
hình.
 - c. Bây giờ hãy thử thêm Batch Normalization và so sánh đường cong học tập: liệu nó có
hội tụ nhanh hơn trước không? Liệu nó có tạo ra mô hình tốt hơn không? Nó ảnh
hưởng đến tốc độ huấn luyện như thế nào?
 - d. Hãy thử thay thế Batch Normalization bằng SELU và thực hiện...
Các điều chỉnh cần thiết để đảm bảo mạng tự chuẩn hóa (ví dụ: chuẩn hóa các đặc trưng
đầu vào, sử dụng khởi tạo chuẩn LeCun, đảm bảo mạng nơ-ron sâu chỉ chứa một chuỗi
các lớp dày đặc, v.v.).
 - e. Hãy thử điều chỉnh mô hình bằng alpha dropout. Sau đó, mà không cần huấn luyện
lại mô hình, hãy xem liệu bạn có thể đạt được độ chính xác tốt hơn bằng cách sử
dụng MC Dropout hay không.

f. Hãy huấn luyện lại mô hình của bạn bằng cách sử dụng phương pháp lập lịch 1 chu kỳ và xem liệu kết quả có được cải thiện hay không.

Tốc độ huấn luyện và độ chính xác của mô hình.

Đáp án cho các bài tập này có sẵn ở cuối tập tài liệu của chương này, tại địa chỉ <https://homl.info/colab3>.

-
- 1 Xavier Glorot và Yoshua Bengio, "Hiểu được khó khăn của việc huấn luyện sâu sắc" "Mạng nơ-ron truyền thẳng", Kỷ yếu Hội nghị quốc tế lần thứ 13 về Trí tuệ nhân tạo và Thống kê (2010): 249-256.
 2. Đây là một ví dụ tương tự: nếu bạn vặn núm điều chỉnh của bộ khuếch đại micro quá gần mức 0, mọi người sẽ không nghe thấy giọng bạn, nhưng nếu bạn vặn quá gần mức tối đa, giọng bạn sẽ bị méo tiếng và mọi người sẽ không hiểu bạn đang nói gì. Bây giờ hãy tưởng tượng một chuỗi các bộ khuếch đại như vậy: tất cả chúng đều cần được thiết lập đúng cách để giọng nói của bạn phát ra to và rõ ràng ở cuối chuỗi. Giọng nói của bạn phải phát ra từ mỗi bộ khuếch đại với biên độ bằng với biên độ khi nó đi vào.
 3. Ví dụ: Kaiming He và cộng sự, "Nghiên cứu sâu về bộ chỉnh lưu: Vượt qua trình độ con người" Hiệu suất phân loại trên ImageNet," Kỷ yếu Hội nghị Quốc tế IEEE về Thị giác Máy tính năm 2015 (2015): 1026-1034.
 4. Một tế bào thần kinh đã chết có thể sống lại nếu các tín hiệu đầu vào của nó thay đổi theo thời gian và cuối cùng quay trở lại. trong một phạm vi mà hàm kích hoạt ReLU nhận được đầu vào dương trở lại. Ví dụ, điều này có thể xảy ra nếu thuật toán Gradient Descent điều chỉnh các nơ-ron trong các lớp bên dưới nơ-ron đã chết.
 - 5 Bing Xu và cộng sự, "Đánh giá thực nghiệm về các kích hoạt được hiệu chỉnh trong mạng tích chập," bản thảo arXiv arXiv:1505.00853 (2015).
 - 6 Djork-Arné Clevert và cộng sự, "Học mạng sâu nhanh và chính xác bằng các đơn vị tuyến tính theo cấp số mũ (ELU)," Kỷ yếu Hội nghị quốc tế về Biểu diễn học tập (2016).
 - 7 Günter Klambauer và cộng sự, "Mạng thần kinh tự chuẩn hóa", Kỷ yếu Hội nghị quốc tế lần thứ 31 về Hệ thống xử lý thông tin thần kinh (2017): 972-981.
 - 8 Dan Hendrycks và Kevin Gimpel, "Các đơn vị tuyến tính lỗi Gaussian (GELUs)," bản thảo arXiv arXiv:1606.08415 (2016).
 9. Một hàm số được gọi là lỗi nếu đoạn thẳng nối hai điểm bất kỳ trên đường cong đó không bao giờ nằm dưới một đường cong nào. Đường cong. Hàm đơn điệu chỉ tăng hoặc chỉ giảm.
 - 10 Prajit Ramachandran và cộng sự, "Tìm kiếm các hàm kích hoạt," bản thảo arXiv arXiv:1710.05941 (2017).
 - 11 Diganta Misra, "Mish: Một hàm kích hoạt phi đơn điệu tự điều chỉnh." Bản thảo arXiv arXiv:1908.08681 (2019).
 - 12 Sergey Ioffe và Christian Szegedy, "Chuẩn hóa hàng loạt: Tăng tốc quá trình huấn luyện mạng sâu bằng cách giảm sự dịch chuyển hiệp biến nội bộ," Kỷ yếu Hội nghị Quốc tế lần thứ 32 về Học máy (2015): 448-456.

- 13 Tuy nhiên, chúng được ước tính trong quá trình huấn luyện, dựa trên dữ liệu huấn luyện, vì vậy có thể nói chúng là có thể huấn luyện được. Trong Keras, "không thể huấn luyện được" thực sự có nghĩa là "không bị ảnh hưởng bởi lan truyền ngược".
- 14 Razvan Pascanu và cộng sự, "Về độ khó của việc huấn luyện mạng nơ-ron hồi quy," Kỷ yếu Hội nghị Quốc tế lần thứ 30 về Học máy (2013): 1310-1318.
- 15 Boris T. Polyak, "Một số phương pháp tăng tốc độ hội tụ của các phương pháp lặp," Toán học tính toán và Vật lý toán học Liên Xô 4, số 5 (1964): 1-17.
- 16 Yurii Nesterov, "Một phương pháp cho bài toán tối thiểu hóa lỗi không ràng buộc với tốc độ hội tụ $O(1/k^2)$," Doklady AN Liên Xô 269 (1983): 543-547.
- 17 John Duchi và cộng sự, "Phương pháp hạ gradient thích ứng cho học trực tuyến và tối ưu hóa ngẫu nhiên," Tạp chí Nghiên cứu Học máy 12 (2011): 2121-2159.
- 18 Thuật toán này được tạo ra bởi Geoffrey Hinton và Tijmen Tieleman vào năm 2012 và được Geoffrey Hinton trình bày trong lớp học Coursera của ông về mạng nơ-ron (slide: <https://homl.info/57>; video: <https://homl.info/58>). Điều thú vị là, vì các tác giả không viết bài báo nào để mô tả thuật toán, nên các nhà nghiên cứu thường trích dẫn "trang chiếu 29 trong bài giảng 6e" trong các bài báo của họ.
- 19 ρ là chữ cái Hy Lạp rho.
- 20 Diederik P. Kingma và Jimmy Ba, "Adam: Một phương pháp tối ưu hóa ngẫu nhiên," arXiv Bản thảo trước khi xuất bản arXiv:1412.6980 (2014).
- 21 Timothy Dozat, "Kết hợp động lượng Nesterov vào Adam" (2016).
- 22 Ilya Loshchilov và Frank Hutter. "Điều chỉnh giảm trọng lượng tách rời." Bản thảo arXiv arXiv:1711.05101 (2017).
- 23 Ashia C. Wilson et al., "Giá trị cận biên của các phương pháp gradient thích ứng trong học máy," Tiến bộ trong hệ thống xử lý thông tin thần kinh 30 (2017): 4148-4158.
- 24 Leslie N. Smith, "Một cách tiếp cận có kỷ luật đối với các siêu tham số của mạng nơ-ron: Phần 1- "Tốc độ học, kích thước lô, động lượng và sự suy giảm trọng số," bản thảo arXiv arXiv:1803.09820 (2018).
- 25 Andrew Senior và cộng sự, "Nghiên cứu thực nghiệm về tốc độ học trong mạng nơ-ron sâu cho Nhận dạng giọng nói," Kỷ yếu Hội nghị Quốc tế IEEE về Âm học, Giọng nói và Xử lý Tín hiệu (2013): 6724-6728.
- 26 Geoffrey E. Hinton và cộng sự, "Cải thiện mạng nơ-ron bằng cách ngăn chặn sự thích nghi đồng thời của "Bộ dò đặc điểm," bản thảo arXiv arXiv:1207.0580 (2012).
- 27 Nitish Srivastava et al., "Dropout: Một cách đơn giản để ngăn chặn mạng nơ-ron bị quá khớp," Tạp chí Nghiên cứu Học máy 15 (2014): 1929-1958.
- 28 Yarin Gal và Zoubin Ghahramani, "Dropout như một phép xấp xỉ Bayes: Biểu diễn sự không chắc chắn của mô hình trong học sâu," Kỷ yếu Hội nghị Quốc tế lần thứ 33 về Học máy (2016): 1050-1059.
- 29 Cụ thể, họ chỉ ra rằng việc huấn luyện mạng dropout tương đương về mặt toán học với suy luận Bayes xấp xỉ trong một loại mô hình xác suất cụ thể được gọi là Quy trình Gauss sâu (Deep Gaussian Process).

30 Lớp MCDropout này sẽ hoạt động với tất cả các API của Keras, bao gồm cả API Sequential. Nếu bạn chỉ quan tâm đến API Functional hoặc API Subclassing, bạn không cần tạo lớp MCDropout; bạn có thể tạo một lớp Dropout thông thường và gọi nó với `training=True`.

[OceanofPDF.com](https://oceanofpdf.com)

Chương 12. Mô hình tùy chỉnh và huấn luyện với TensorFlow

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 12 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Cho đến nay, chúng ta chỉ sử dụng API cấp cao của TensorFlow, Keras, nhưng nó đã giúp chúng ta tiến khá xa: chúng ta đã xây dựng nhiều kiến trúc mạng nơ-ron khác nhau, bao gồm mạng hồi quy và phân loại, mạng Wide & Deep, và mạng tự chuẩn hóa, sử dụng đủ loại kỹ thuật, chẳng hạn như Batch Normalization, dropout và learning rate schedules. Trên thực tế, 95% các trường hợp sử dụng mà bạn sẽ gặp phải sẽ không yêu cầu bất cứ thứ gì khác ngoài Keras (và tf.data; xem [Chương 13](#)). Nhưng bây giờ là lúc để tìm hiểu sâu hơn về TensorFlow và xem xét [API Python cấp thấp hơn của nó](#). Điều này sẽ hữu ích khi bạn cần thêm quyền kiểm soát để viết các hàm mất mát tùy chỉnh, các chỉ số tùy chỉnh, các lớp, mô hình, bộ khởi tạo, bộ điều chỉnh, ràng buộc trọng số, và nhiều hơn nữa. Bạn thậm chí có thể cần kiểm soát hoàn toàn vòng lặp huấn luyện, ví dụ như để áp dụng các phép biến đổi hoặc ràng buộc đặc biệt cho gradient (ngoài việc chỉ cắt bớt chúng) hoặc để sử dụng nhiều bộ tối ưu hóa cho các phần khác nhau của mạng. Chúng ta sẽ đề cập đến tất cả các trường hợp này trong chương này, và chúng ta cũng sẽ xem xét cách bạn có thể tăng cường các mô hình và thuật toán huấn luyện tùy chỉnh của mình bằng cách sử dụng TensorFlow.

Tính năng tạo đồ thị tự động. Nhưng trước tiên, hãy cùng tìm hiểu nhanh về TensorFlow.

Tổng quan nhanh về TensorFlow

Như bạn đã biết, TensorFlow là một thư viện mạnh mẽ dành cho tính toán số học, đặc biệt phù hợp và được tinh chỉnh tốt cho Học máy quy mô lớn (nhưng bạn cũng có thể sử dụng nó cho bất kỳ ứng dụng nào khác yêu cầu tính toán phức tạp). Nó được phát triển bởi nhóm Google Brain và là nền tảng cho nhiều dịch vụ quy mô lớn của Google, chẳng hạn như Google Cloud Speech, Google Photos và Google Search. Nó được công khai mã nguồn vào tháng 11 năm 2015, và hiện là thư viện Học sâu được sử dụng rộng rãi nhất trong ngành: vô số dự án sử dụng TensorFlow cho tất cả các loại nhiệm vụ Học máy¹, chẳng hạn như phân loại hình ảnh, xử lý ngôn ngữ tự nhiên, hệ thống đề xuất và dự báo chuỗi thời gian.

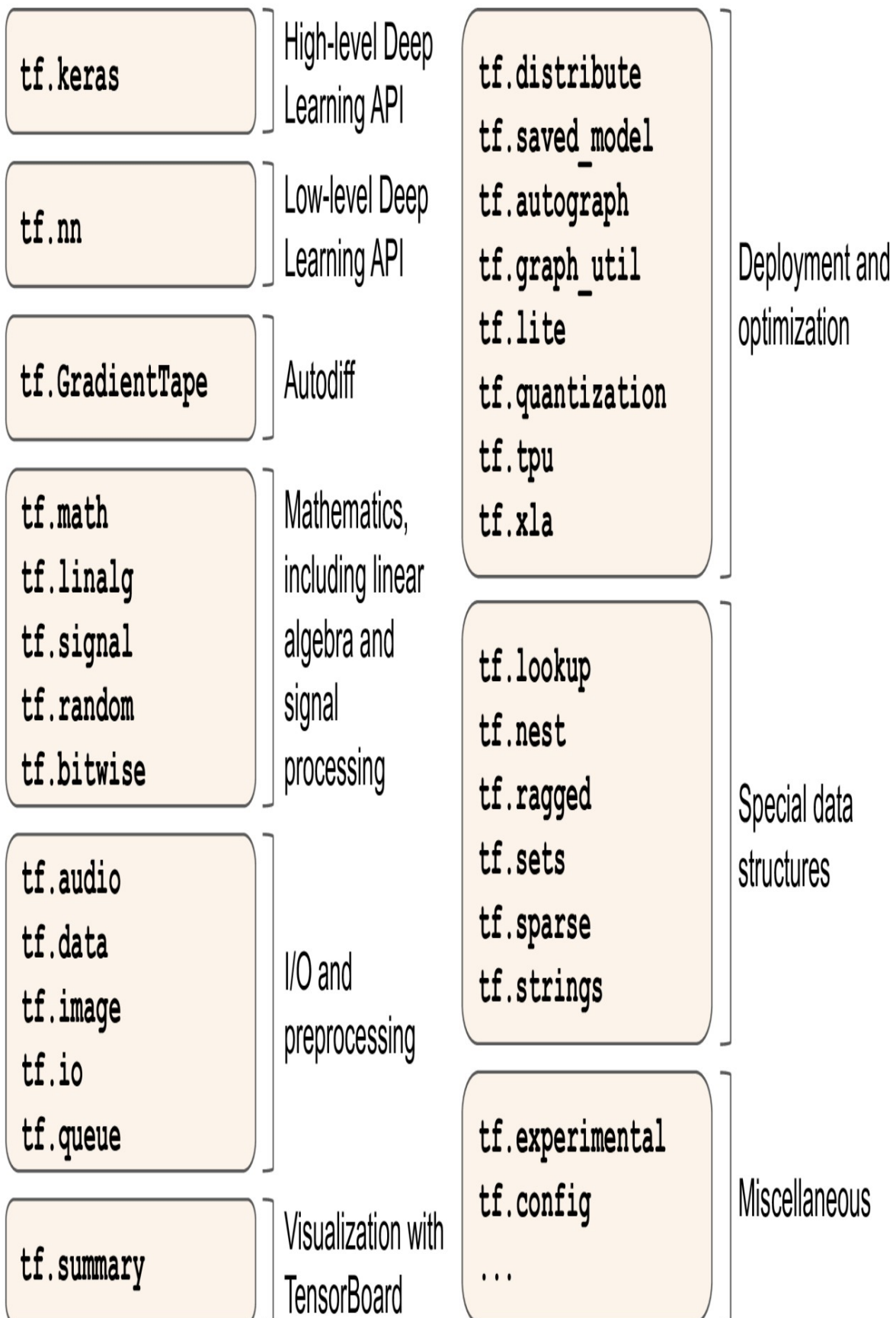
Vậy TensorFlow mang lại những gì? Dưới đây là tóm tắt:

- Về cơ bản, nó có cốt lõi rất giống với NumPy, nhưng có hỗ trợ GPU.
- Nó hỗ trợ điện toán phân tán (trên nhiều thiết bị và máy chủ).
- Nó bao gồm một loại trình biên dịch tức thời (JIT) cho phép tối ưu hóa các phép tính về tốc độ và mức sử dụng bộ nhớ. Nó hoạt động bằng cách trích xuất đồ thị tính toán từ một hàm Python, sau đó tối ưu hóa nó (ví dụ: bằng cách loại bỏ các nút không sử dụng), và cuối cùng chạy nó một cách hiệu quả (ví dụ: bằng cách tự động chạy các phép toán độc lập song song).
- Đồ thị tính toán có thể được xuất sang định dạng di động, vì vậy bạn có thể huấn luyện mô hình TensorFlow trong một môi trường (ví dụ: sử dụng Python trên Linux) và chạy nó trong một môi trường khác (ví dụ: sử dụng Java trên thiết bị Android).
- Nó triển khai phương pháp tự động tính toán ngược (xem [Chương 10](#) và [Phụ lục B](#)) và cung cấp một số thuật toán tối ưu hóa xuất sắc, chẳng hạn như RMSProp và Nadam (xem [Chương 11](#)), giúp bạn dễ dàng tối thiểu hóa tất cả các loại hàm mất mát.

TensorFlow cung cấp nhiều tính năng hơn nữa được xây dựng dựa trên các tính năng cốt lõi này: quan trọng nhất tất nhiên là Keras², nhưng nó cũng có các thao tác tải và tiền xử lý dữ liệu (`tf.data`, `tf.io`, v.v.), các thao tác xử lý hình ảnh (`tf.image`), các thao tác xử lý tín hiệu (`tf.signal`), và nhiều hơn nữa (xem [Hình 12-1](#) để có cái nhìn tổng quan về API Python của TensorFlow).

MẸO

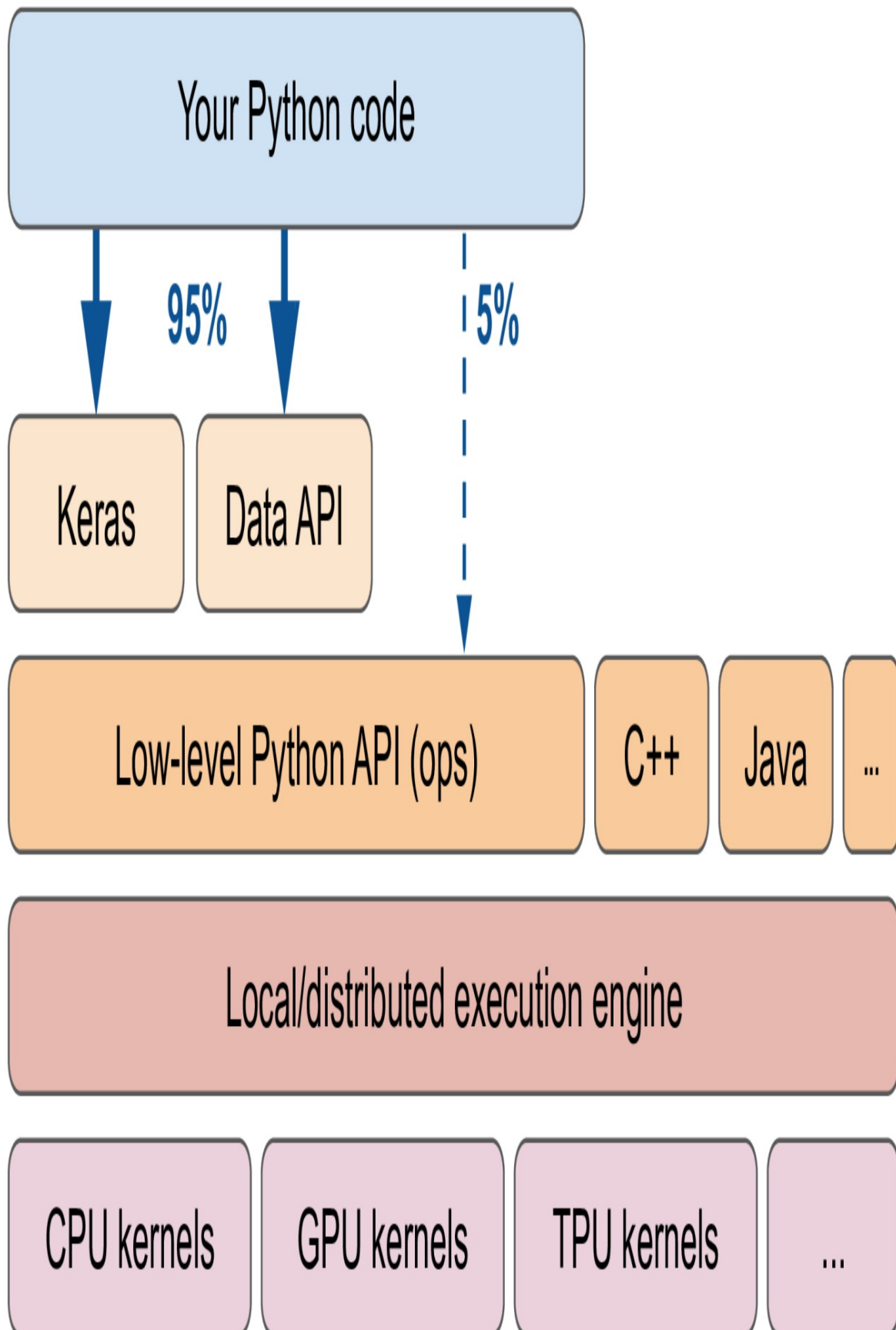
Chúng ta sẽ tìm hiểu nhiều gói và hàm của API TensorFlow, nhưng không thể bao quát hết tất cả, vì vậy bạn thực sự nên dành thời gian để xem qua API; bạn sẽ thấy rằng nó khá phong phú và được tài liệu hóa tốt.



Hình 12-1. API Python của TensorFlow

Ở cấp độ thấp nhất, mỗi thao tác TensorFlow (viết tắt là op) được triển khai bằng mã C++ hiệu quả cao. Nhiều thao tác³ có nhiều cách triển khai được gọi là kernel: mỗi kernel dành riêng cho một loại thiết bị cụ thể, chẳng hạn như CPU, GPU hoặc thậm chí là TPU (bộ xử lý tensor). Như bạn đã biết, GPU có thể tăng tốc đáng kể các phép tính bằng cách chia chúng thành nhiều phần nhỏ hơn và chạy song song trên nhiều luồng GPU. TPU thậm chí còn nhanh hơn: chúng là các chip ASIC tùy chỉnh được xây dựng đặc biệt cho các hoạt động Học sâu (chúng ta sẽ thảo luận về cách sử dụng TensorFlow với GPU hoặc TPU trong **Chương 19**).

Kiến trúc của TensorFlow được thể hiện trong **Hình 12-2**. Hầu hết thời gian, mã của bạn sẽ sử dụng các API cấp cao (đặc biệt là Keras và tf.data); nhưng khi cần linh hoạt hơn, bạn sẽ sử dụng API Python cấp thấp hơn, xử lý trực tiếp các tensor. Trong mọi trường hợp, công cụ thực thi của TensorFlow sẽ đảm bảo chạy các thao tác một cách hiệu quả, ngay cả trên nhiều thiết bị và máy tính khác nhau nếu bạn yêu cầu.



Hình 12-2. Kiến trúc của TensorFlow

TensorFlow không chỉ chạy trên Windows, Linux và macOS, mà còn trên các thiết bị di động (sử dụng TensorFlow Lite), bao gồm cả iOS và Android (xem [Chương 19](#)). Lưu ý rằng các API cho các ngôn ngữ khác cũng có sẵn nếu bạn không muốn sử dụng API Python: có các API C++, Java và Swift. Thậm chí còn có một phiên bản JavaScript có tên TensorFlow.js cho phép bạn chạy mô hình trực tiếp trong trình duyệt.

TensorFlow không chỉ đơn thuần là thư viện. TensorFlow là trung tâm của một hệ sinh thái thư viện rộng lớn. Đầu tiên, có TensorBoard để trực quan hóa (xem [Chương 10](#)). Tiếp theo, có [TensorFlow Extended \(TFX\)](#), TensorFlow Hub là một tập hợp các thư viện do Google xây dựng để đưa các dự án TensorFlow vào sản xuất: nó bao gồm các công cụ để xác thực dữ liệu, tiền xử lý, phân tích mô hình và triển khai (với TF Serving; xem [Chương 19](#)). TensorFlow Hub của Google cung cấp một cách dễ dàng tải xuống và tái sử dụng các mạng nơ-ron đã được huấn luyện trước. Bạn cũng có thể tìm thấy nhiều kiến trúc mạng nơ-ron, một số trong đó đã được huấn luyện trước, trong [kho mô hình của TensorFlow](#). Hãy xem [phần Tài nguyên TensorFlow](#) và <https://github.com/jtoy/awesome-tensorflow> Để tìm thêm các dự án dựa trên TensorFlow, bạn có thể tìm thấy hàng trăm dự án TensorFlow trên GitHub, vì vậy việc tìm kiếm mã nguồn hiện có cho bất cứ điều gì bạn đang cố gắng thực hiện thường rất dễ dàng.

MẸO

Ngày càng có nhiều bài báo về học máy được công bố cùng với các triển khai của chúng, và đôi khi thậm chí cả các mô hình được huấn luyện trước. Hãy xem <https://paperswithcode.com/> Để dễ dàng tìm thấy chúng.

Cuối cùng nhưng không kém phần quan trọng, TensorFlow có một đội ngũ phát triển tận tâm, nhiệt huyết và luôn sẵn sàng hỗ trợ, cũng như một cộng đồng lớn đóng góp vào việc cải tiến nó. Để đặt câu hỏi kỹ thuật, bạn nên sử dụng <https://stackoverflow.com/> Và hãy gắn thẻ câu hỏi của bạn với từ khóa "tensorflow" và "python". Bạn có thể báo cáo lỗi và yêu cầu tính năng thông qua [GitHub](#). Để thảo luận chung, hãy tham gia Diễn đàn [TensorFlow](#) .

Được rồi, đến lúc bắt đầu lập trình thôi!

Sử dụng TensorFlow tương tự như NumPy

API của TensorFlow xoay quanh các tensor, chúng truyền từ phép toán này sang phép toán khác – do đó có tên là TensorFlow. Một tensor rất giống với một ndarray của NumPy: nó thường là một mảng đa chiều, nhưng nó cũng có thể chứa một giá trị vô hướng (một giá trị đơn giản, chẳng hạn như 42). Các tensor này sẽ rất quan trọng khi chúng ta tạo các hàm chi phí tùy chỉnh, các chỉ số tùy chỉnh, các lớp tùy chỉnh, v.v., vì vậy hãy cùng xem cách tạo và thao tác với chúng.

Tensor và các phép toán

Bạn có thể tạo một tensor bằng `tf.constant()`. Ví dụ, đây là một tensor biểu diễn một ma trận có hai hàng và ba cột gồm các số thực:

```
>>> import tensorflow as tf >>> t =
tf.constant([[1., 2., 3.], [4., 5., 6.]]) # ma trận >>> t <tf.Tensor: shape=(2, 3),
dtype=float32, numpy= array([[1., 2., 3.], [4., 5., 6.]], dtype=float32)>
```

Giống như ndarray, `tf.Tensor` có hình dạng và kiểu dữ liệu (`dtype`):

```
>>> t.shape
TensorShape([2, 3]) >>>
t.dtype tf.float32
```

Việc lập chỉ mục hoạt động tương tự như trong NumPy:

```
>>> t[:, 1:]
<tf.Tensor: shape=(2, 2), dtype=float32, numpy= array([[2., 3.], [5.,
6.]], dtype=float32)>
>>> t[..., 1, tf.newaxis]
```

```
<tf.Tensor: shape=(2, 1), dtype=float32, numpy= array([[2.],
[5.]],
dtype=float32)>
```

Quan trọng nhất, tất cả các loại phép toán tensor đều có sẵn:

```
>>> t + 10
<tf.Tensor: shape=(2, 3), dtype=float32, numpy= array([[11.,
12., 13.], [14., 15., 16.]],
dtype=float32)> >>> tf.square(t) <tf.Tensor:
shape=(2, 3),
dtype=float32, numpy= array([[ 1.,  4.,  9.], [16., 25., 36.]],
dtype=float32)> >>> t @
tf.transpose(t) <tf.Tensor: shape=(2, 2),
dtype=float32, numpy=
array([[14., 32.], [32., 77.]], dtype=float32)>
```

Lưu ý rằng việc viết `t + 10` tương đương với việc gọi `tf.add(t, 10)` (thực tế, Python gọi phương thức đặc biệt `t.__add__(10)`, thực chất chỉ gọi `tf.add(t, 10)`). Các toán tử khác như `-` và toán `*` cũng được hỗ trợ. Toán tử `@` được thêm vào trong Python 3.5, dùng cho phép nhân ma trận: nó tương đương với việc gọi hàm `tf.matmul()`.

GHI CHÚ

Nhiều hàm và lớp có bí danh. Ví dụ, `tf.add()` và `tf.math.add()` là cùng một hàm. Điều này cho phép TensorFlow có cách viết ngắn gọn hơn. Đặt tên cho các thao tác phổ biến nhất đồng thời vẫn giữ được sự ngăn nắp của các gói.

Một tensor cũng có thể chứa một giá trị vô hướng. Trong trường hợp này, hình dạng của nó là rỗng:

```
>>> tf.constant(42)
<tf.Tensor: shape=(), dtype=int32, numpy=42>
```

Bạn sẽ tìm thấy tất cả các phép toán cơ bản cần thiết (`tf.add()`, `tf.multiply()`, `tf.square()`, `tf.exp()`, `tf.sqrt()`, v.v.) và

Hầu hết các phép toán mà bạn có thể tìm thấy trong NumPy (ví dụ: `tf.reshape()`, `tf.squeeze()`, `tf.tile()`). Một số hàm có tên khác so với trong NumPy; chẳng hạn, `tf.reduce_mean()`, `tf.reduce_sum()`, `tf.reduce_max()` và `tf.math.log()` tương đương với `np.mean()`, `np.sum()`, `np.max()` và `np.log()`.

Khi tên gọi khác nhau, thường có lý do chính đáng. Ví dụ, trong TensorFlow bạn phải viết `tf.transpose(t)`; bạn không thể chỉ viết `tT` như trong NumPy. Lý do là hàm `tf.transpose()` không thực hiện chính xác cùng một việc như thuộc tính `T` của NumPy: trong TensorFlow, một tensor mới được tạo ra với bản sao riêng của dữ liệu đã được chuyển vị, trong khi ở NumPy, `tT` chỉ là một dạng chuyển vị trên cùng dữ liệu đó. Tương tự, phép toán `tf.reduce_sum()` được đặt tên như vậy vì nhân GPU của nó (tức là triển khai trên GPU) sử dụng thuật toán giảm không đảm bảo thứ tự cộng các phần tử: vì số thực 32 bit có độ chính xác hạn chế, kết quả có thể thay đổi một chút mỗi khi bạn gọi phép toán này. Điều tương tự cũng đúng với `tf.reduce_mean()` (nhưng tất nhiên `tf.reduce_max()` là xác định).

GHI CHÚ

API của Keras có API cấp thấp riêng, nằm trong gói `tf.keras.backend`. Gói này thường được nhập dưới dạng `K` để ngắn gọn. Trước đây, nó bao gồm các hàm như `K.square()`, `K.exp()` và `K.sqrt()`, mà bạn có thể bắt gặp trong mã hiện có: điều này hữu ích để viết mã có tính di động khi Keras hỗ trợ nhiều backend, nhưng hiện tại Keras chỉ hỗ trợ TensorFlow, bạn nên gọi trực tiếp API cấp thấp của TensorFlow (ví dụ: `tf.square()` thay vì `K.square()`). Về mặt kỹ thuật, `K.square()` và các hàm tương tự vẫn còn đó để đảm bảo khả năng tương thích ngược, nhưng tài liệu của gói `tf.keras.backend` chỉ liệt kê một số ít hàm tiện ích, chẳng hạn như `clear_session()` (đã thảo luận trong [Chương 10](#)).

Tensor và NumPy

Tensor tương thích tốt với NumPy: bạn có thể tạo tensor từ mảng NumPy và ngược lại.

Bạn thậm chí có thể áp dụng các phép toán TensorFlow cho mảng NumPy và các phép toán NumPy cho tensor:

```
>>> import numpy as np
>>> a = np.array([2., 4., 5.])
>>> tf.constant(a)
<tf.Tensor: id=111, shape=(3,), dtype=float64, numpy=array([2., 4.,
5.])> >>>
t.numpy() # hoặc np.array(t)
array([[1., 2., 3.],
       [4., 5., 6.]], dtype=float32)
>>> tf.square(a)
<tf.Tensor: id=116, shape=(3,), dtype=float64, numpy=array([4., 16.,
25.])> >>>
np.square(t)
array([[ 1.,  4.,  9.],
       [16., 25., 36.]], dtype=float32)
```

CẢNH BÁO

Lưu ý rằng NumPy sử dụng độ chính xác 64 bit theo mặc định, trong khi TensorFlow sử dụng 32 bit. Điều này là do độ chính xác 32 bit thường là quá đủ cho mạng nơ-ron, cộng thêm nó chạy nhanh hơn và sử dụng ít RAM hơn. Vì vậy, khi bạn tạo một tensor từ mảng NumPy, hãy đảm bảo đặt dtype=tf.float32.

Chuyển đổi kiểu dữ liệu

Việc chuyển đổi kiểu dữ liệu có thể ảnh hưởng đáng kể đến hiệu năng và rất dễ bị bỏ qua khi chúng được thực hiện tự động. Để tránh điều này, TensorFlow không thực hiện bất kỳ chuyển đổi kiểu dữ liệu nào tự động: nó chỉ báo lỗi nếu bạn cố gắng thực hiện một phép toán trên các tensor có kiểu dữ liệu không tương thích. Ví dụ, bạn không thể cộng một tensor kiểu float và một tensor kiểu integer, và thậm chí bạn cũng không thể cộng một float 32 bit và một float 64 bit:

```
>>> tf.constant(2.) + tf.constant(40)
[...] InvalidArgumentError: [...] được mong đợi là một tensor float
[...]
>>> tf.constant(2.) + tf.constant(40., dtype=tf.float64)
[...] InvalidArgumentError: [...] được mong đợi là một tensor float
[...]
```

Ban đầu điều này có thể hơi khó chịu, nhưng hãy nhớ rằng đó là vì một mục đích tốt đẹp! Và tất nhiên bạn có thể sử dụng tf.cast() khi thực sự cần chuyển đổi.

các loại:

```
>>> t2 = tf.constant(40., dtype=tf.float64) >>>
tf.constant(2.0) + tf.cast(t2, tf.float32) <tf.Tensor:
id=136, shape=(), dtype=float32, numpy=42.0>
```

Biến số

Các giá trị `tf.Tensor` mà chúng ta đã thấy cho đến nay là bất biến: bạn không thể sửa đổi chúng. Điều này có nghĩa là chúng ta không thể sử dụng các tensor thông thường để triển khai trọng số trong mạng nơ-ron, vì chúng cần được điều chỉnh bằng thuật toán lan truyền ngược. Thêm vào đó, các tham số khác cũng có thể cần thay đổi theo thời gian (ví dụ: bộ tối ưu hóa động lượng theo dõi các gradient trong quá khứ). Cái chúng ta cần là một `tf.Variable`:

```
>>> v = tf.Variable([[1., 2., 3.], [4., 5., 6.]])
>>> v
<tf.Variable 'Variable:0' shape=(2, 3) dtype=float32, numpy= array([[1., 2.,
3.], [4., 5., 6.]],
      dtype=float32)>
```

Một `tf.Variable` hoạt động khá giống với `tf.Tensor`: bạn có thể thực hiện các phép toán tương tự với nó, nó cũng hoạt động tốt với NumPy và cũng kén chọn kiểu dữ liệu. Nhưng nó cũng có thể được sửa đổi trực tiếp bằng phương thức `assign()` (hoặc `assign_add()` hoặc `assign_sub()`, tăng hoặc giảm biến theo giá trị được cung cấp). Bạn cũng có thể sửa đổi từng ô (hoặc lát cắt) riêng lẻ bằng cách sử dụng phương thức `assign()` của ô (hoặc lát cắt) hoặc bằng cách sử dụng các phương thức `scatter_update()` hoặc `scatter_nd_update()`:

```
v.assign(2 * v)                # v bây giờ bằng [[2., 4., 6.], [8., 10.,
12.]]
v[0, 1].assign(42) 10.,        # v bây giờ bằng [[2., 42., 6.], [8.,
12.]] v[:,
2].assign([0., 1.]) # v bây giờ bằng [[2., 42., 0.], [8., 10., 1.]]

v.scatter_nd_update( 10.,      # v bây giờ bằng [[100., 42., 0.], [8.,
200.]]
    chỉ số=[[0, 0], [1, 2]], cập nhật=[100., 200.] )
```

Việc gán trực tiếp sẽ không hiệu quả:

```
>>> v[1] = [7., 8., 9.]  
[...] TypeError: Đối tượng 'ResourceVariable' không hỗ trợ gán mục
```

GHI CHÚ

Trên thực tế, bạn sẽ hiếm khi phải tạo biến thủ công, vì Keras cung cấp phương thức `add_weight()` sẽ tự động thực hiện việc đó cho bạn, như chúng ta sẽ thấy. Hơn nữa, các tham số của mô hình thường sẽ được cập nhật trực tiếp bởi các trình tối ưu hóa, vì vậy bạn sẽ hiếm khi cần cập nhật biến thủ công.

Các cấu trúc dữ liệu khác

TensorFlow hỗ trợ một số cấu trúc dữ liệu khác, bao gồm các cấu trúc sau (vui lòng xem phần “Các cấu trúc dữ liệu khác” trong sổ tay hoặc

(Xem thêm chi tiết [tại Phụ lục C](#)):

Tensor thưa (`tf.SparseTensor`)

Biểu diễn hiệu quả các tensor chứa chủ yếu là số 0. Gói `tf.sparse` chứa các phép toán dành cho tensor thưa.

Mảng tensor (`tf.TensorArray`)

Đây là các danh sách tensor. Chúng có độ dài cố định theo mặc định nhưng có thể tùy chọn mở rộng. Tất cả các tensor trong đó phải có cùng hình dạng và kiểu dữ liệu.

Tensor không đều (`tf.RaggedTensor`)

Biểu diễn danh sách các tensor, tất cả đều có cùng hạng và kiểu dữ liệu, nhưng kích thước khác nhau. Các chiều mà kích thước tensor thay đổi được gọi là các chiều không đều (`ragged dimensions`). Gói `tf.ragged` chứa các phép toán cho các tensor không đều.

Tensor chuỗi

Đây là các tensor thông thường thuộc loại `tf.string`. Chúng đại diện cho các chuỗi byte, không phải chuỗi Unicode, vì vậy nếu bạn tạo một tensor chuỗi bằng cách sử dụng một chuỗi Unicode (ví dụ: một chuỗi Python 3 thông thường như "café"), thì nó sẽ được mã hóa thành UTF-8 một cách tự động (ví dụ: `b"caf\xc3\xa9"`). Ngoài ra, bạn cũng có thể biểu diễn chuỗi Unicode bằng cách sử dụng tensor kiểu `tf.int32`, trong đó mỗi phần tử đại diện cho một điểm mã Unicode (ví dụ: `[99, 97, 102, 233]`). Gói `tf.strings` (có thêm hậu tố `s`) chứa các phép toán cho chuỗi byte và chuỗi Unicode (và để chuyển đổi từ loại này sang loại khác). Điều quan trọng cần lưu ý là `tf.string` là nguyên tử, nghĩa là độ dài của nó không xuất hiện trong hình dạng của tensor. Sau khi bạn chuyển đổi nó thành một tensor Unicode (tức là một tensor kiểu `tf.int32` chứa các điểm mã Unicode), độ dài sẽ xuất hiện trong hình dạng.

Bộ

Chúng được biểu diễn dưới dạng tensor thông thường (hoặc tensor thưa). Ví dụ, `tf.constant([[1, 2], [3, 4]])` biểu diễn hai tập hợp `{1, 2}` và `{3, 4}`. Tổng quát hơn, mỗi tập hợp được biểu diễn bằng một vectơ trên trục cuối cùng của tensor. Bạn có thể thao tác với các tập hợp bằng cách sử dụng các phép toán từ gói `tf.sets`.

Hàng đợi

Lưu trữ tensor qua nhiều bước. TensorFlow cung cấp nhiều loại hàng đợi khác nhau: hàng đợi cơ bản vào trước ra trước (FIFO) (`FIFOQueue`), hàng đợi có thể ưu tiên một số mục (`PriorityQueue`), xáo trộn các mục của chúng (`RandomShuffleQueue`) và nhóm các mục có kích thước khác nhau bằng cách thêm phân đệm (`PaddingFIFOQueue`). Tất cả các lớp này đều nằm trong gói `tf.queue`.

Với các tensor, phép toán, biến và nhiều cấu trúc dữ liệu khác nhau trong tay, giờ đây bạn đã sẵn sàng tùy chỉnh các mô hình và thuật toán huấn luyện của mình!

Tùy chỉnh mô hình và thuật toán huấn luyện

Chúng ta hãy bắt đầu bằng cách tạo một hàm mất mát tùy chỉnh, đây là một trường hợp sử dụng đơn giản và phổ biến.

Hàm mất mát tùy chỉnh

Giả sử bạn muốn huấn luyện một mô hình hồi quy, nhưng tập dữ liệu huấn luyện của bạn hơi nhiễu. Tất nhiên, bạn bắt đầu bằng cách cố gắng làm sạch tập dữ liệu bằng cách loại bỏ hoặc sửa các giá trị ngoại lệ, nhưng điều đó hóa ra là không đủ; tập dữ liệu vẫn còn nhiễu. Bạn nên sử dụng hàm mất mát nào? Sai số bình phương trung bình (MSE) có thể phạt quá nhiều các lỗi lớn và khiến mô hình của bạn không chính xác. Sai số tuyệt đối trung bình (MAE) sẽ không phạt các giá trị ngoại lệ nhiều, nhưng quá trình huấn luyện có thể mất nhiều thời gian để hội tụ, và mô hình được huấn luyện có thể không chính xác lắm.

Đây có lẽ là thời điểm thích hợp để sử dụng hàm mất mát Huber (được giới thiệu trong [Chương 10](#)) thay vì hàm mất mát MSE quen thuộc. Hàm mất mát Huber có sẵn trong Keras (chỉ cần sử dụng một thẻ hiện của lớp `tf.keras.losses.Huber`).

Nhưng hãy giả vờ như nó không tồn tại. Để triển khai nó, chỉ cần tạo một hàm nhận nhãn và dự đoán của mô hình làm đối số, và sử dụng các phép toán TensorFlow để tính toán một tensor chứa tất cả các tổn thất (một tổn thất cho mỗi mẫu):

```
def huber_fn(y_true, y_pred): error = y_true -  
    y_pred is_small_error = tf.abs(error)  
    < 1 squared_loss = tf.square(error) / 2 linear_loss =  
    tf.abs(error) - 0.5 return tf.where(is_small_error,  
    squared_loss, linear_loss)
```

CẢNH BÁO

Để đạt hiệu suất tốt hơn, bạn nên sử dụng phương pháp triển khai vector hóa, như trong ví dụ này. Hơn nữa, nếu bạn muốn tận dụng các tính năng tối ưu hóa đồ thị của TensorFlow, bạn chỉ nên sử dụng các phép toán TensorFlow.

Cũng có thể trả về giá trị tổn thất trung bình thay vì tổn thất mẫu riêng lẻ, nhưng điều này không được khuyến khích vì nó khiến việc sử dụng trọng số lớp hoặc trọng số mẫu trở nên không thể thực hiện được khi cần thiết (xem [Chương 10](#)).

Giờ bạn có thể sử dụng hàm mất mát Huber này khi biên dịch mô hình Keras, sau đó huấn luyện mô hình như bình thường:

```
model.compile(loss=huber_fn, optimizer="nadam") model.fit(X_train,
y_train, [...])
```

Và đó là tất cả! Trong mỗi batch trong quá trình huấn luyện, Keras sẽ gọi hàm `huber_fn()` để tính toán tổn thất, sau đó sử dụng phương pháp tự động tính toán ngược (reverse-mode autodiff) để tính toán đạo hàm của tổn thất đối với tất cả các tham số của mô hình, và cuối cùng thực hiện bước Gradient Descent (trong ví dụ này sử dụng trình tối ưu hóa Nadam). Hơn nữa, nó sẽ theo dõi tổng tổn thất kể từ đầu epoch và hiển thị tổn thất trung bình.

Nhưng điều gì sẽ xảy ra với phần mất mát tùy chỉnh này khi bạn lưu mô hình?

Lưu và tải các mô hình chứa nội dung tùy chỉnh

Các thành phần

Việc lưu mô hình chứa hàm mất mát tùy chỉnh hoạt động tốt, nhưng khi tải mô hình đó, bạn cần cung cấp một từ điển ánh xạ tên hàm đến hàm thực tế. Nói chung hơn, khi tải mô hình chứa các đối tượng tùy chỉnh, bạn cần ánh xạ tên đến các đối tượng:

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss", custom_objects={"huber_fn":
huber_fn})
```

MẸO

Nếu bạn trang trí hàm `huber_fn()` bằng `@keras.utils.register_keras_serializable()`, nó sẽ tự động khả dụng cho hàm `load_model()`: không cần phải thêm nó vào từ điển `custom_objects`.

Với cách triển khai hiện tại, bất kỳ sai số nào nằm giữa -1 và 1 đều được coi là "nhỏ". Nhưng nếu bạn muốn một ngưỡng khác thì sao? Một giải pháp là tạo ra một hàm tạo ra hàm mất mát được cấu hình:

```
def create_huber(threshold=1.0):
    def huber_fn(y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < threshold
        squared_loss = tf.square(error) / 2
        linear_loss = threshold * tf.abs(error) - threshold ** 2
        return tf.where(is_small_error, squared_loss, linear_loss)
    return huber_fn

model.compile(loss=create_huber(2.0), optimizer="nadam")
```

Thật không may, khi bạn lưu mô hình, ngưỡng sẽ không được lưu lại. Điều này có nghĩa là bạn sẽ phải chỉ định giá trị ngưỡng khi tải mô hình (lưu ý rằng tên cần sử dụng là "huber_fn", đây là tên của hàm bạn đã cung cấp cho Keras, chứ không phải tên của hàm đã tạo ra nó):

```
model = tf.keras.models.load_model("my_model_with_a_custom_loss_threshold_2", custom_objects={
    "huber_fn": huber_fn
})
```

Bạn có thể giải quyết vấn đề này bằng cách tạo một lớp con của lớp `tf.keras.losses.Loss`, và sau đó triển khai phương thức `get_config()` của lớp con đó:

```
class HuberLoss(tf.keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold ** 2
        return tf.where(is_small_error, squared_loss, linear_loss)
```

```

        linear_loss = self.threshold * tf.abs(error) -
        Ngưỡng tự thân**2 / 2
        return tf.where(is_small_error, squared_loss, linear_loss)

    def get_config(self):
        base_config = super().get_config() return
        {**base_config, "threshold": self.threshold}

```

Hãy cùng phân tích đoạn mã này:

- Hàm khởi tạo chấp nhận các đối số `**kwargs` và truyền chúng cho hàm khởi tạo cha, hàm này xử lý các siêu tham số chuẩn: tên của hàm mất mát và thuật toán giảm được sử dụng để tổng hợp các hàm mất mát của từng trường hợp riêng lẻ. Theo mặc định, nó là "AUTO", tương đương với "SUM_OVER_BATCH_SIZE": hàm mất mát sẽ là tổng của các hàm mất mát của từng trường hợp, được trọng số hóa bằng trọng số mẫu (nếu có) và chia cho kích thước lô (không phải chia cho tổng trọng số, vì vậy đây không phải là trung bình có trọng số). Các giá trị khả dĩ khác là "SUM" và "NONE".⁶
- Phương thức `call()` nhận các nhãn và dự đoán, tính toán tất cả các tổn thất của từng trường hợp và trả về chúng.
- Phương thức `get_config()` trả về một từ điển ánh xạ tên của mỗi siêu tham số với giá trị tương ứng. Đầu tiên, nó gọi phương thức `get_config()` của lớp cha, sau đó thêm các siêu tham số mới vào từ điển này.

⁷

Sau đó, bạn có thể sử dụng bất kỳ thể hiện nào của lớp này khi biên dịch mô hình:

```
model.compile(loss=HuberLoss(2.), optimizer="nadam")
```

Khi bạn lưu mô hình, ngưỡng cũng sẽ được lưu cùng với nó; và khi bạn tải mô hình, bạn chỉ cần ánh xạ tên lớp với chính lớp đó:

```

model =
tf.keras.models.load_model("my_model_with_a_custom_loss_class",

```

```

custom_objects={"HuberLoss":
HuberLoss})

```

Khi bạn lưu một mô hình, Keras gọi phương thức `get_config()` của thể hiện `loss` và lưu cấu hình vào `SavedModel`. Khi bạn tải mô hình, nó gọi phương thức `from_config()` trên lớp `HuberLoss`: phương thức này được triển khai bởi lớp cơ sở (`Loss`) và tạo một thể hiện của lớp, truyền `**config` vào hàm tạo.

Vậy là xong phần về tổn thất! Như chúng ta sẽ thấy ngay bây giờ, các hàm kích hoạt tùy chỉnh, hàm khởi tạo, hàm điều chỉnh và các ràng buộc không khác nhau nhiều.

Các hàm kích hoạt, hàm khởi tạo, hàm điều chỉnh và các ràng buộc tùy chỉnh

Hầu hết các chức năng của Keras, chẳng hạn như hàm mất mát, bộ điều chỉnh, ràng buộc, bộ khởi tạo, số liệu, hàm kích hoạt, các lớp và thậm chí cả toàn bộ mô hình, đều có thể được tùy chỉnh theo cùng một cách. Thông thường, bạn chỉ cần viết một hàm đơn giản với các đầu vào và đầu ra phù hợp. Dưới đây là các ví dụ về hàm kích hoạt tùy chỉnh (tương đương với `tf.keras.activations.softplus()` hoặc `tf.nn.softplus()`), bộ khởi tạo Glorot tùy chỉnh (tương đương với `tf.keras.initializers.glorot_normal()`), bộ điều chỉnh ℓ_1 tùy chỉnh (tương đương với `tf.keras.regularizers.l1(0.01)`) và ràng buộc tùy chỉnh đảm bảo tất cả các trọng số đều dương (tương đương với `tf.keras.constraints.nonneg()` hoặc `tf.nn.relu()`):

```

def my_softplus(z):
    return tf.math.log(1.0 + tf.exp(z))

def my_glorot_initializer(shape, dtype=tf.float32):
    stddev = tf.sqrt(2. / (shape[0] + shape[1]))
    return tf.random.normal(shape, stddev=stddev, dtype=dtype)

def my_l1_regularizer(weights):
    return tf.reduce_sum(tf.abs(0.01 * weights))

def my_positive_weights(weights): # giá trị trả về chỉ là tf.nn.relu(weights)

```

```
return tf.where(weights < 0., tf.zeros_like(weights), weights)
```

Như bạn thấy, các tham số phụ thuộc vào loại hàm tùy chỉnh.

Sau đó, các hàm tùy chỉnh này có thể được sử dụng bình thường; ví dụ:

```
lớp = tf.keras.layers.Dense(1, activation=my_softplus,
                             kernel_initializer=my_glorot_initializer,
                             kernel_regularizer=my_l1_regularizer,
                             kernel_constraint=my_positive_weights)
```

Hàm kích hoạt sẽ được áp dụng cho đầu ra của lớp Dense này, và kết quả của nó sẽ được truyền đến lớp tiếp theo. Trọng số của lớp sẽ được khởi tạo bằng giá trị trả về từ hàm khởi tạo. Ở mỗi bước huấn luyện, trọng số sẽ được truyền đến hàm điều chỉnh để tính toán tổn thất điều chỉnh, tổn thất này sẽ được cộng vào tổn thất chính để có được tổn thất cuối cùng được sử dụng cho quá trình huấn luyện. Cuối cùng, hàm ràng buộc sẽ được gọi sau mỗi bước huấn luyện, và trọng số của lớp sẽ được thay thế bằng trọng số bị ràng buộc.

Nếu một hàm có các siêu tham số cần được lưu cùng với mô hình, thì bạn sẽ muốn tạo lớp con của lớp phù hợp, chẳng hạn như

```
tf.keras.regularizers.Regularizer,
tf.keras.constraints.Constraint,
tf.keras.initializers.Initializer hoặc
tf.keras.layers.Layer (cho bất kỳ lớp nào, bao gồm cả các hàm kích
hoạt). Tương tự như cách chúng ta đã làm với hàm mất mát tùy chỉnh, đây là một
lớp đơn giản cho chuẩn hóa & lưu siêu tham số factor của nó (lần này chúng
ta không cần gọi hàm tạo của lớp cha hoặc phương thức get_config(), vì chúng
không được định nghĩa bởi lớp cha):
```

```
class MyL1Regularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, factor):
        self.factor = factor
```

```
def __call__(self, weights):
    return tf.reduce_sum(tf.abs(self.factor * weights))

def get_config(self):
    return {"factor": self.factor}
```

Lưu ý rằng bạn phải triển khai phương thức `__call__()` cho các hàm mất mát, các lớp (bao gồm cả hàm kích hoạt) và các mô hình, hoặc phương thức `__call__()` cho các bộ điều chỉnh, bộ khởi tạo và các ràng buộc. Đối với các chỉ số đo lường, mọi thứ sẽ khác một chút, như chúng ta sẽ thấy ngay sau đây.

Số liệu tùy chỉnh

Về mặt khái niệm, hàm mất mát và các chỉ số đánh giá không giống nhau: hàm mất mát (ví dụ: entropy chéo) được thuật toán Gradient Descent sử dụng để huấn luyện mô hình, vì vậy chúng phải khả vi (ít nhất là tại các điểm được đánh giá), và đạo hàm của chúng không được bằng 0 ở mọi nơi. Thêm vào đó, việc chúng không dễ hiểu đối với con người cũng không sao. Ngược lại, các chỉ số đánh giá (ví dụ: độ chính xác) được sử dụng để đánh giá mô hình: chúng phải dễ hiểu hơn, và chúng có thể không khả vi hoặc có đạo hàm bằng 0 ở mọi nơi.

Tuy nhiên, trong hầu hết các trường hợp, việc định nghĩa một hàm đo lường tùy chỉnh hoàn toàn giống với việc định nghĩa một hàm mất mát tùy chỉnh. Trên thực tế, chúng ta thậm chí có thể sử dụng hàm mất mát Huber mà chúng ta đã tạo trước đó làm một hàm đo lường; nó sẽ hoạt động tốt (và việc lưu trữ cũng sẽ hoạt động theo cách tương tự, trong trường hợp này chỉ lưu tên của hàm, "huber_fn", chứ không phải ngưỡng):

```
model.compile(loss="mse", optimizer="nadam", metrics=[create_huber(2.0)])
```

Trong mỗi lô dữ liệu khi huấn luyện, Keras sẽ tính toán chỉ số này và theo dõi giá trị trung bình của nó kể từ đầu kỷ nguyên. Hầu hết thời gian, đây chính xác là những gì bạn muốn. Nhưng không phải lúc nào cũng vậy! Ví dụ, hãy xem xét độ chính xác của một bộ phân loại nhị phân. Như chúng ta đã thấy trong [Chương 3](#), độ chính xác là số lượng kết quả dương tính thực sự chia cho số lượng dự đoán dương tính (bao gồm cả kết quả dương tính thực sự và kết quả dương tính giả). Giả sử mô hình đưa ra năm dự đoán dương tính trong lô đầu tiên, trong đó bốn dự đoán là chính xác: đó là 80%.

độ chính xác. Giả sử mô hình đưa ra ba dự đoán tích cực trong lô thứ hai, nhưng tất cả đều không chính xác: đó là độ chính xác 0% cho lô thứ hai. Nếu bạn chỉ tính trung bình của hai độ chính xác này, bạn sẽ nhận được 40%.

Nhưng khoan đã—đó không phải là độ chính xác của mô hình trên hai lô dữ liệu này!

Thực tế, có tổng cộng bốn kết quả dương tính thật ($4 + 0$) trong số tám dự đoán dương tính ($5 + 3$), vì vậy độ chính xác tổng thể là 50%, chứ không phải 40%. Điều chúng ta cần là một đối tượng có thể theo dõi số lượng kết quả dương tính thật và số lượng kết quả dương tính giả, đồng thời có thể tính toán độ chính xác dựa trên các con số này khi được yêu cầu. Đây chính xác là những gì lớp `tf.keras.metrics.Precision` thực hiện:

```
>>> precision = tf.keras.metrics.Precision()
>>> precision([0, 1, 1, 1, 0, 1, 0, 1], [1, 1, 0, 1, 0, 1, 0, 1])
<tf.Tensor: shape=(), dtype=float32, numpy=0.8>
>>> precision([0, 1, 0, 0, 1, 0, 1, 1], [1, 0, 1, 1, 0, 0, 0, 0])
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
```

Trong ví dụ này, chúng ta đã tạo một đối tượng `Precision`, sau đó sử dụng nó như một hàm, truyền vào đó các nhãn và dự đoán cho lô đầu tiên, rồi cho lô thứ hai (bạn cũng có thể tùy chọn truyền trọng số mẫu nếu muốn).

Chúng tôi đã sử dụng cùng số lượng kết quả dương tính thật và dương tính giả như trong ví dụ vừa thảo luận. Sau lô đầu tiên, nó trả về độ chính xác 80%; sau đó, sau lô thứ hai, nó trả về 50% (đây là độ chính xác tổng thể cho đến nay, chứ không phải độ chính xác của lô thứ hai). Điều này được gọi là số liệu theo luồng (hoặc số liệu có trạng thái), vì nó được cập nhật dần dần, từng lô một.

Tại bất kỳ thời điểm nào, chúng ta đều có thể gọi phương thức `result()` để lấy giá trị hiện tại của chỉ số. Chúng ta cũng có thể xem các biến của nó (theo dõi số lượng kết quả đúng và sai) bằng cách sử dụng thuộc tính `variables`, và chúng ta có thể đặt lại các biến này bằng phương thức `reset_states()`:

```
>>> precision.result()
<tf.Tensor: shape=(), dtype=float32, numpy=0.5>
>>> precision.variables
[<tf.Variable 'true_positives:0' [...], numpy=array([4.],
dtype=float32)>,
 <tf.Variable 'false_positives:0' [...], numpy=array([4.],
```

```
dtype=float32)>] >>>
precision.reset_states() # cả hai biến đều được đặt lại về 0.0
```

Nếu bạn cần định nghĩa chỉ số đo lường luồng dữ liệu tùy chỉnh của riêng mình, hãy tạo một lớp con của lớp `tf.keras.metrics.Metric`. Dưới đây là một ví dụ cơ bản theo dõi tổng tổn thất Huber và số lượng trường hợp đã được quan sát cho đến nay. Khi được yêu cầu trả về kết quả, nó sẽ trả về tỷ lệ, đó chính là giá trị trung bình của tổn thất Huber:

```
class HuberMetric(tf.keras.metrics.Metric):
    def __init__(self, threshold=1.0, **kwargs):
        super().__init__(**kwargs) # xử lý các đối số cơ bản (ví dụ:
        kiểu dữ liệu)
        self.threshold = threshold
        self.huber_fn = create_huber(threshold) self.total =
        self.add_weight("total",
        khởi tạo="số không")
        self.count = self.add_weight("count",
        khởi tạo="số không")

    def update_state(self, y_true, y_pred, sample_weight=None): sample_metrics =
        self.huber_fn(y_true, y_pred)
        self.total.assign_add(tf.reduce_sum(sample_metrics))
        self.count.assign_add(tf.cast(tf.size(y_true), tf.float32))

    def result(self):
        return self.total / self.count

    def get_config(self):
        base_config = super().get_config() return
        {**base_config, "threshold": self.threshold}
```

Hãy cùng phân tích đoạn mã này: ⁹

- Hàm khởi tạo sử dụng phương thức `add_weight()` để tạo các biến cần thiết nhằm theo dõi trạng thái của chỉ số qua nhiều lô – trong trường hợp này, tổng của tất cả các tổn thất Huber (total) và số lượng trường hợp đã thấy cho đến nay (count). Bạn cũng có thể tự tạo các biến nếu muốn. Keras theo dõi bất kỳ `tf.Variable` nào.

đó được thiết lập như một thuộc tính (và nói chung hơn, bất kỳ đối tượng nào có thể "theo dõi được", chẳng hạn như các lớp hoặc mô hình).

- Phương thức `update_state()` được gọi khi bạn sử dụng một thể hiện của lớp này như một hàm (như chúng ta đã làm với đối tượng `Precision`). Nó cập nhật các biến, dựa trên nhãn và dự đoán cho một lô (và trọng số mẫu, nhưng trong trường hợp này chúng ta bỏ qua chúng).
- Phương thức `result()` tính toán và trả về kết quả cuối cùng, trong trường hợp này là chỉ số Huber trung bình trên tất cả các trường hợp. Khi bạn sử dụng chỉ số này như một hàm, phương thức `update_state()` sẽ được gọi trước, sau đó phương thức `result()` được gọi và kết quả đầu ra của nó được trả về.
- Chúng tôi cũng triển khai phương thức `get_config()` để đảm bảo ngưỡng được lưu cùng với mô hình.
- Phương thức `reset_states()` được triển khai mặc định sẽ đặt lại tất cả các biến về 0.0 (nhưng bạn có thể ghi đè lên nếu cần).

GHI CHÚ

Keras sẽ tự động xử lý việc lưu trữ biến một cách liền mạch; bạn không cần phải thao tác gì thêm.

Khi bạn định nghĩa một chỉ số bằng một hàm đơn giản, Keras sẽ tự động gọi hàm đó cho mỗi batch và theo dõi giá trị trung bình trong mỗi epoch, giống như cách chúng ta đã làm thủ công. Vì vậy, lợi ích duy nhất của lớp `HuberMetric` là ngưỡng sẽ được lưu lại. Nhưng tất nhiên, một số chỉ số, như độ chính xác, không thể đơn giản được tính trung bình trên các batch: trong những trường hợp đó, không có lựa chọn nào khác ngoài việc triển khai một chỉ số xử lý dữ liệu theo luồng.

Giờ đây, khi chúng ta đã xây dựng được hệ thống đo lường trực tuyến, việc xây dựng một lớp dữ liệu tùy chỉnh sẽ trở nên dễ dàng hơn bao giờ hết!

Lớp tùy chỉnh

Đôi khi bạn có thể muốn xây dựng một kiến trúc chứa một lớp đặc biệt mà TensorFlow không cung cấp triển khai mặc định. Hoặc đơn giản là bạn muốn xây dựng một kiến trúc rất lặp đi lặp lại, trong đó một khối các lớp cụ thể được lặp lại nhiều lần, và sẽ thuận tiện hơn nếu coi mỗi khối như một lớp riêng lẻ. Trong những trường hợp như vậy, bạn sẽ cần xây dựng một lớp tùy chỉnh.

Đầu tiên, một số lớp không có trọng số, chẳng hạn như `tf.keras.layers.Flatten` hoặc `tf.keras.layers.ReLU`. Nếu bạn muốn tạo một lớp tùy chỉnh mà không có bất kỳ trọng số nào, lựa chọn đơn giản nhất là viết một hàm và gói nó trong một lớp `tf.keras.layers.Lambda`. Ví dụ, lớp sau sẽ áp dụng hàm mũ cho đầu vào của nó:

```
exponential_layer = tf.keras.layers.Lambda(lambda x: tf.exp(x))
```

Lớp tùy chỉnh này sau đó có thể được sử dụng như bất kỳ lớp nào khác, bằng cách sử dụng API Tuần tự, API Chức năng hoặc API Kế thừa lớp con. Bạn cũng có thể sử dụng nó như một hàm kích hoạt, hoặc bạn có thể sử dụng `activation=tf.exp`. Lớp hàm mũ đôi khi được sử dụng trong lớp đầu ra của mô hình hồi quy khi các giá trị cần dự đoán có thang đo rất khác nhau (ví dụ: 0,001, 10, 1000). Trên thực tế, hàm mũ là một trong những hàm kích hoạt chuẩn trong Keras, vì vậy bạn chỉ cần sử dụng `activation="exponential"`.

Như bạn có thể đoán, để xây dựng một lớp trạng thái tùy chỉnh (tức là một lớp có trọng số), bạn cần tạo một lớp con của lớp `tf.keras.layers.Layer`. Ví dụ, lớp sau đây triển khai một phiên bản đơn giản hóa của lớp `Dense`:

```
class MyDense(tf.keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
```